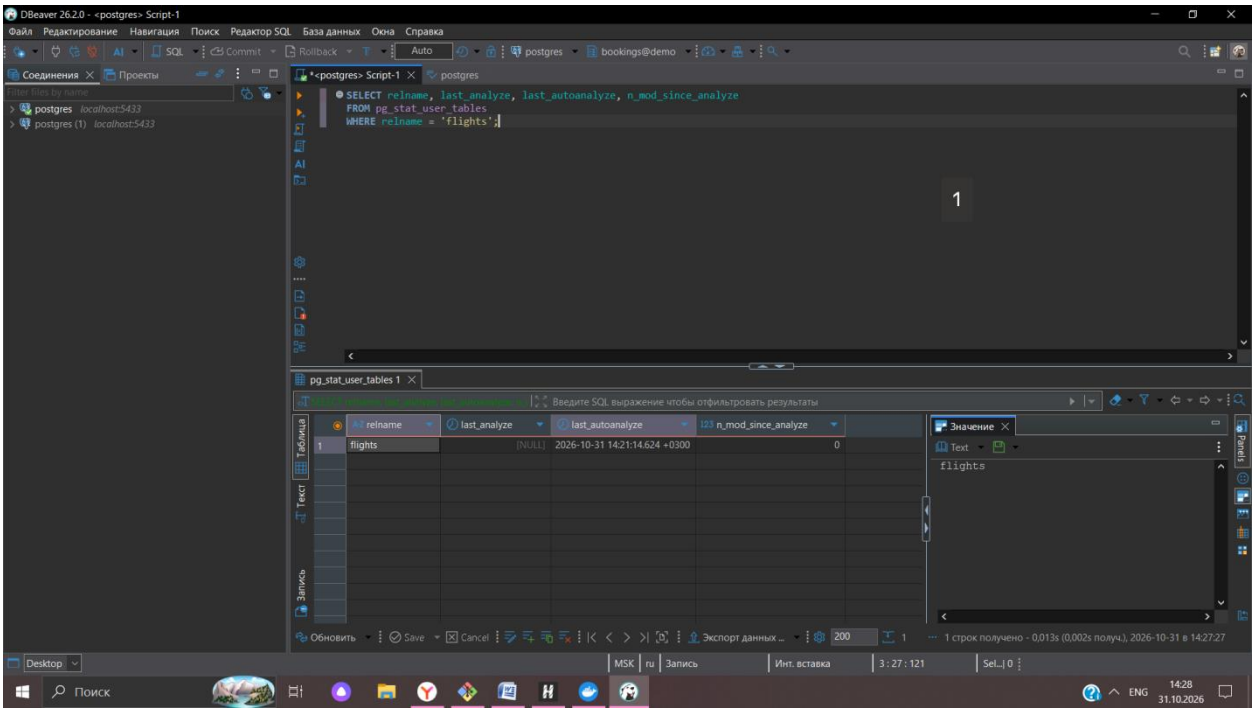
ЗАДАНИЕ 1

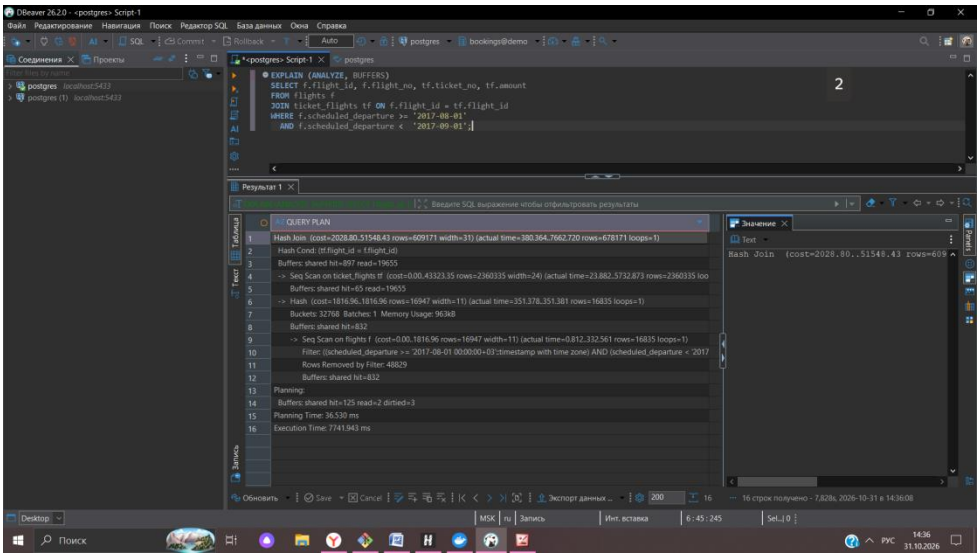


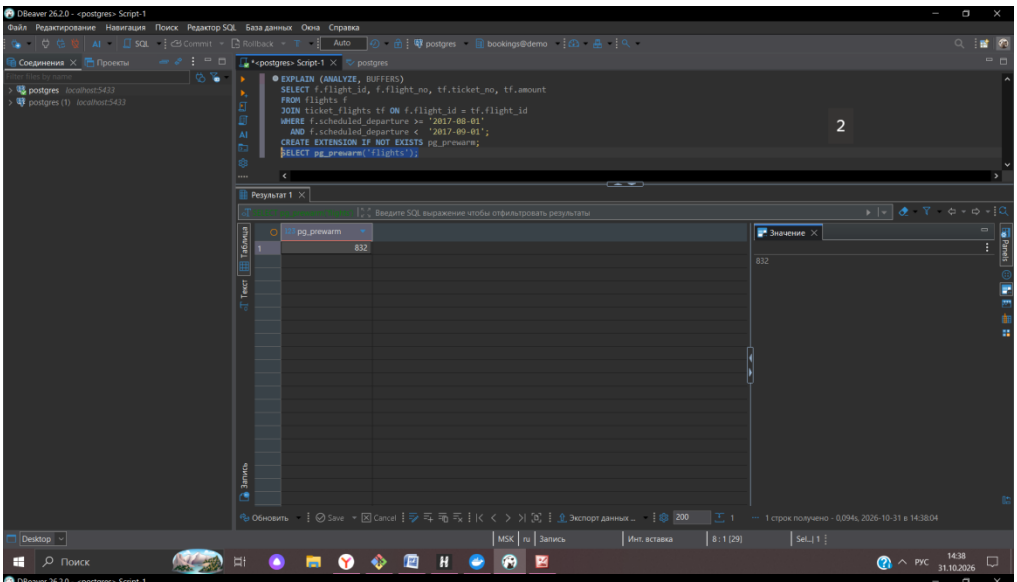
По результату запроса дата последнего ручного анализа `last_analyze` отсутствует (NULL); дата последнего автоматического анализа `last_autoanalyze` — 2026-10-31 14:21:14; количество модификаций `n_mod_since_analyze` — 0.

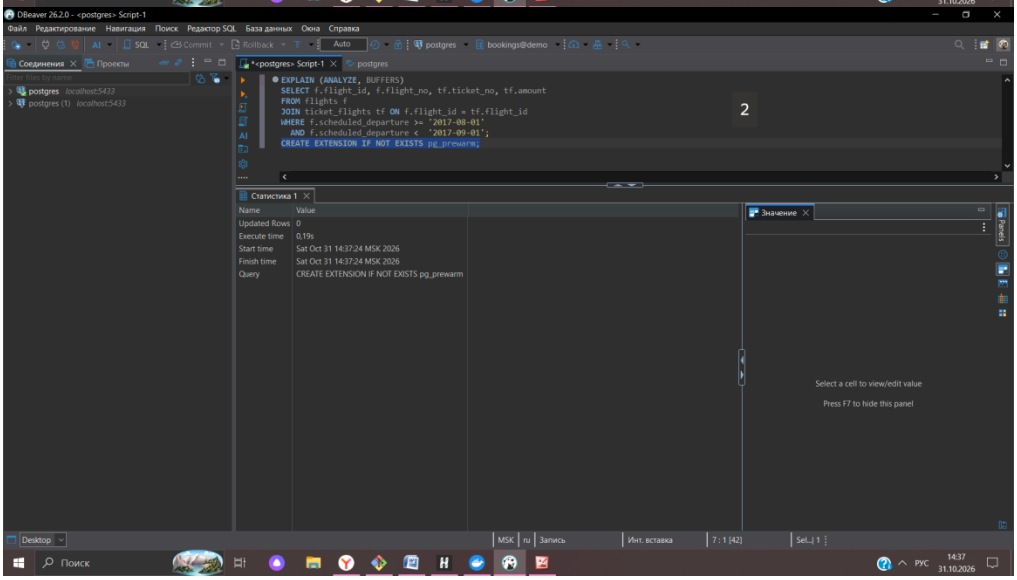
Статистика актуальна: изменений с последнего анализа нет, поэтому порог в 10% от размера таблицы не превышен, и `ANALYZE flights;` не выполнялся.

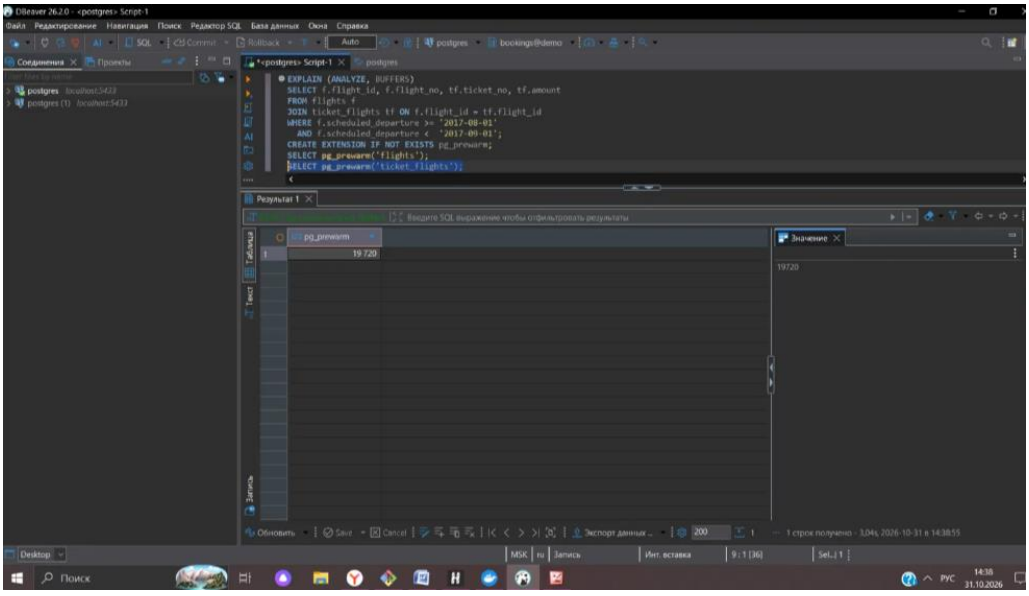
Устаревание статистики определяю по полю `n_mod_since_analyze` (сколько строк изменили после последнего анализа; если их больше 10% от размера таблицы, статистика устарела) и по полям `last_analyze` и `last_autoanalyze` (когда статистику собирали в последний раз вручную и автоматически). После `ANALYZE` ничего не менялось, так как он не запускался.

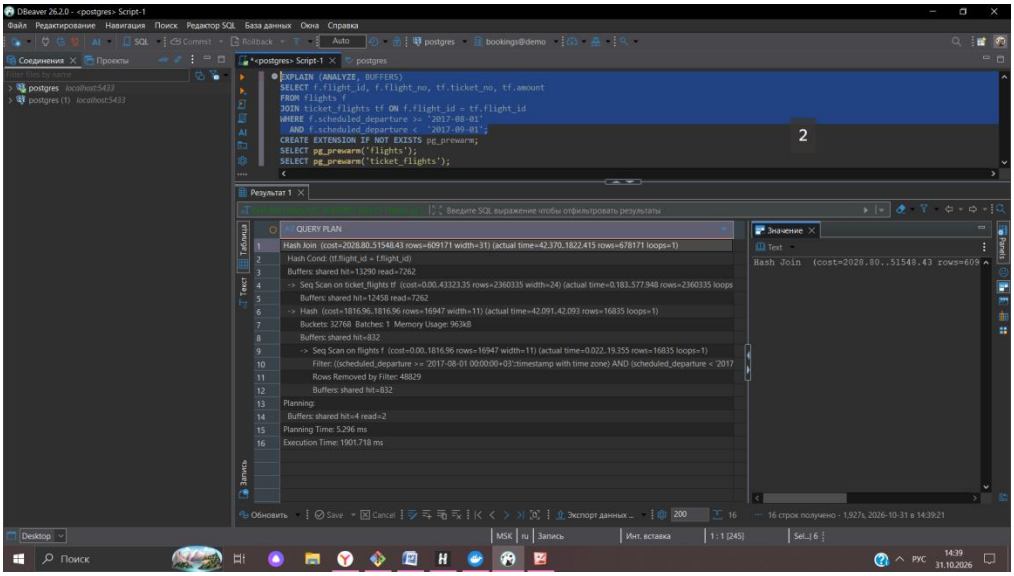
ЗАДАНИЕ 2




DBeaver 26.2.0 - <postgres> Script-1
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_id, f.flight_no, tf.ticket_no, tf.amount
FROM flights f
JOIN ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.scheduled_departure >= '2017-08-01'
AND f.scheduled_departure < '2017-09-01';
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
SELECT pg_prewarm('flights');
2
pg_prewarm
832
Значение
832


DBeaver 26.2.0 - <postgres> Script-1
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_id, f.flight_no, tf.ticket_no, tf.amount
FROM flights f
JOIN ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.scheduled_departure >= '2017-08-01'
AND f.scheduled_departure < '2017-09-01';
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
2
Статистика 1
Name
Value
Updated Rows 0
Execute time 0,19s
Start time Sat Oct 31 14:37:24 MSK 2026
Finish time Sat Oct 31 14:37:24 MSK 2026
Query CREATE EXTENSION IF NOT EXISTS pg_prewarm
Select a cell to view/edit value
Press F7 to hide this panel


DBeaver 26.2.0 - <postgres> Script-1
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_id, f.flight_no, tf.ticket_no, tf.amount
FROM flights f
JOIN ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.scheduled_departure >= '2017-08-01'
AND f.scheduled_departure < '2017-09-01';
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
SELECT pg_prewarm('flights');
SELECT pg_prewarm('ticket_flights');
pg_prewarm
19 720
Значение
19720



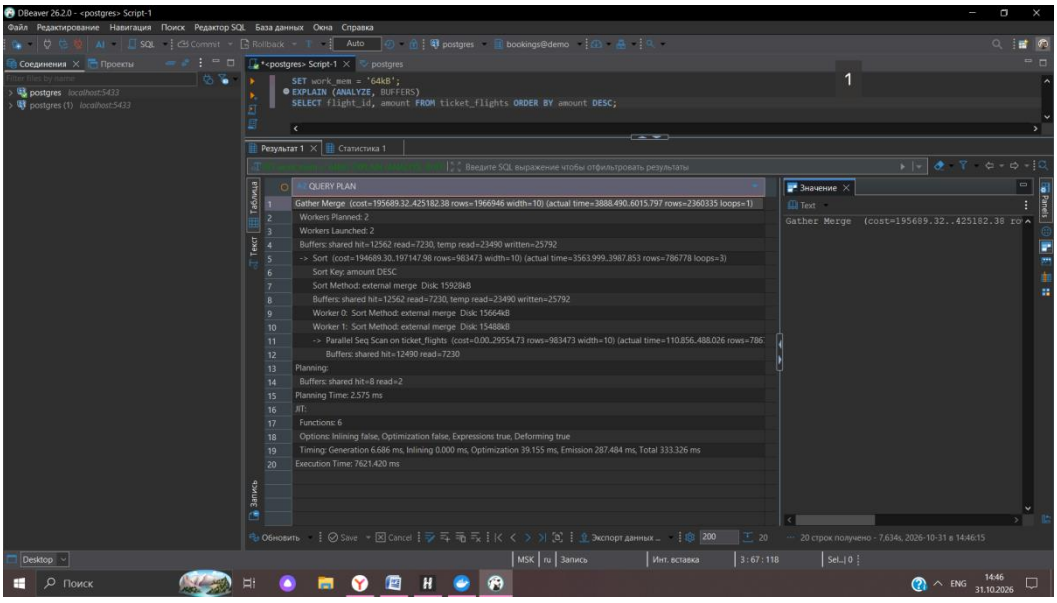
До прогрева План EXPLAIN (ANALYZE, BUFFERS) показывает холодный кэш для ticket_flights: в строке 5 (Seq Scan on ticket_flights) shared hit=65 read=19655, в строке 3 итог по запросу shared hit=897 read=19655. Таблица flights уже была в кэше (строки 8 и 12: shared hit=832). Время выполнения 7741.943 ms.

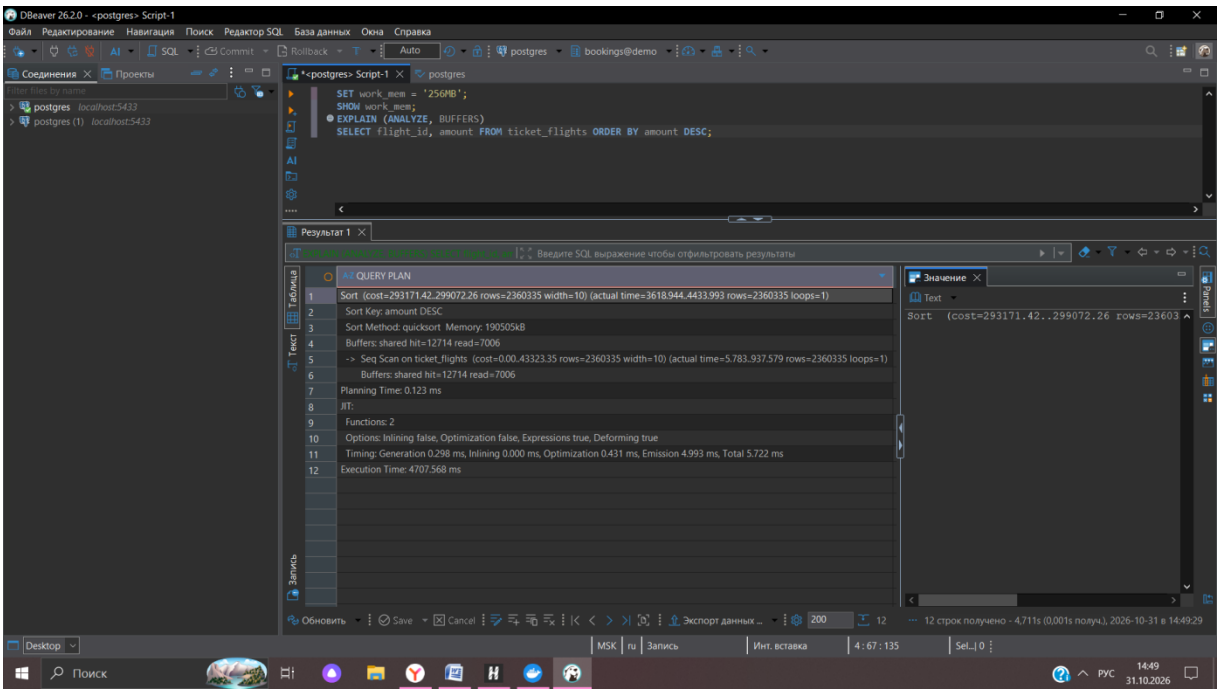
Прогрев. Я создала расширение pg_prewarm и выполнила pg_prewarm для таблиц: flights — загружено 832 блока, ticket_flights — 19 720 блоков.

После прогрева. В строке 5 shared hit=12458 read=7262, в строке 3 shared hit=13290 read=7262. Счётчик shared read уменьшился с 19655 до 7262, а shared hit вырос с 65 до 12458 (строка 5) и с 897 до 13290 (строка 3). Время выполнения сократилось с 7741.943 ms до 1901.718 ms (примерно в 4 раза), чтение ticket_flights ускорилось с 5732.873 ms до 577.948 ms.

Вывод. pg_prewarm заранее загрузил блоки таблиц в буферный кэш shared_buffers, поэтому большую часть данных PostgreSQL взял из памяти (shared hit), а не с диска (shared read). План не изменился (Hash Join и два Seq Scan), поменялись только счётчики буферов и время.

ЗАДАНИЕ 3





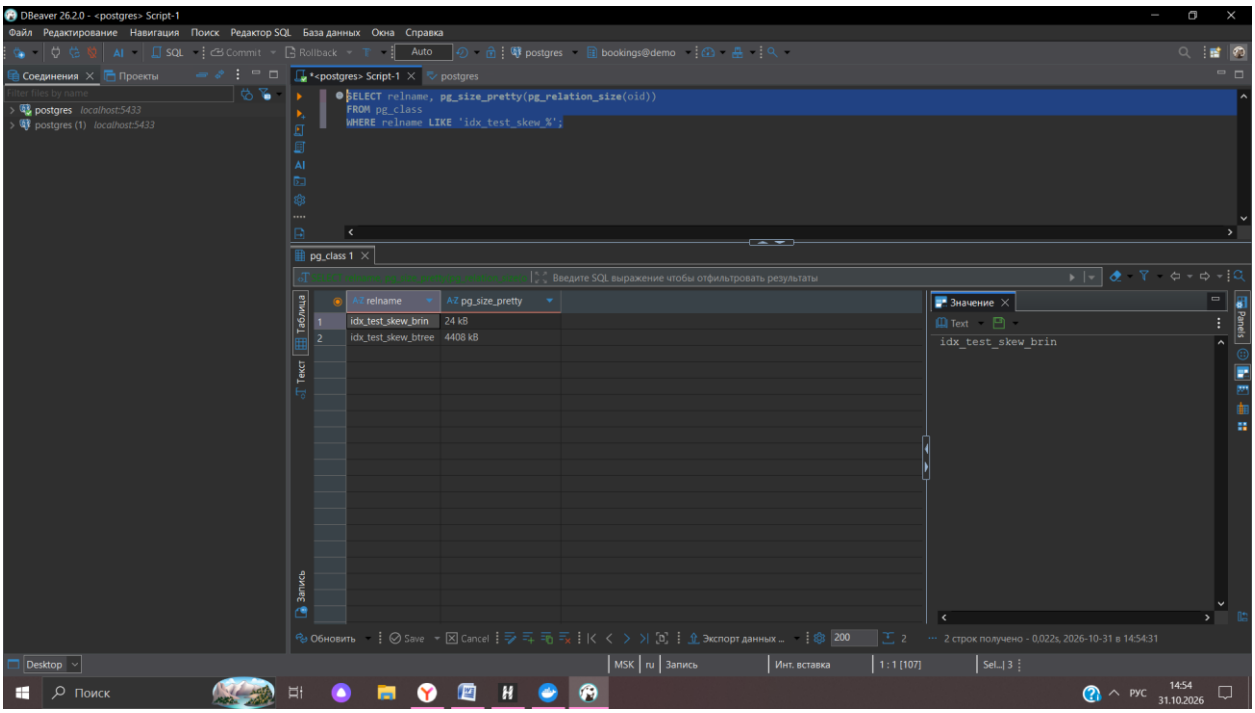
Результат при work_mem = 64kB (скриншот 1). План параллельный (Gather Merge, запущено 2 рабочих процесса). Сортировка ORDER BY amount DESC выполняется внешним слиянием: Sort Method: external merge Disk: 15928kB в основном процессе и Disk: 15664kB и Disk: 15488kB в двух рабочих процессах (у каждого свой work_mem). В Buffers указано temp read=23490 written=25792. Время выполнения: 7621.420 ms**.**

Результат при work_mem = 256MB (скриншот 2). Значение подтверждено командой SHOW work_mem. Сортировка идёт целиком в памяти: Sort Method: quicksort Memory: 190505kB. Метрика Disk и строки temp read/written в плане отсутствуют. Планировщик выбрал непараллельный план (Sort над Seq Scan on ticket_flights), стоимость плана снизилась (с cost=425182.38 до cost=299072.26). Время выполнения: 4707.568 ms**.**

Почему исчезла метрика Disk **и как это повлияло на скорость.** work_mem — объём памяти, который PostgreSQL выделяет одной операции (сортировке, хеш-соединению) в одном запросе. При 64 кБ результат сортировки 2 360 335 строк в память не помещается. PostgreSQL сортирует данные частями, записывает их во временные файлы на диск и затем сливает (external merge, метрика Disk). После увеличения work_mem до 256 МБ все строки (около 186 МБ) помещаются в память, сортировка выполняется целиком в оперативной памяти (quicksort, Memory: 190505kB), временные файлы не создаются, и метрика Disk пропадает. Время выполнения уменьшилось с 7621.420 ms до 4707.568 ms (на 2913.852 ms, примерно на 38%), потому что запись и чтение временных файлов на диске медленнее работы в памяти. Часть времени, которое осталось, уходит на чтение таблицы и саму сортировку.

Почему возникает чтение с диска, если таблица и так в shared_buffers. Это разные области памяти. shared_buffers — общий кэш страниц таблиц и индексов, из него идёт shared hit при чтении самой таблицы ticket_flights. work_mem личная рабочая память одной операции, она нужна для промежуточного результата сортировки, а не для хранения таблицы. Когда этот результат не помещается в work_mem, PostgreSQL сбрасывает его во временные файлы (temp written) и читает обратно (temp read). Поэтому Disk в сортировке означает не чтение таблицы, а работу с временными файлами.

ЗАДАНИЕ 4



Размер индекса idx_test_skew_btree составляет 4408 kB, размер индекса idx_test_skew_brin составляет 24 kB. B-Tree больше BRIN примерно в 184 раза.

B-Tree хранит в дереве запись о каждой строке таблицы: значение ключа и указатель на строку. Поэтому его размер растёт вместе с числом строк: для 100 000 строк получилось 4408 kB.

BRIN делит таблицу на диапазоны страниц и для каждого диапазона хранит только минимальное и максимальное значение столбца. Записей в нём столько, сколько диапазонов, а не строк, поэтому индекс занимает всего 24 kB. Платой за компактность становится меньшая точность: BRIN указывает лишь, в каких диапазонах страниц может находиться значение, а строки внутри диапазона всё равно приходится просматривать.

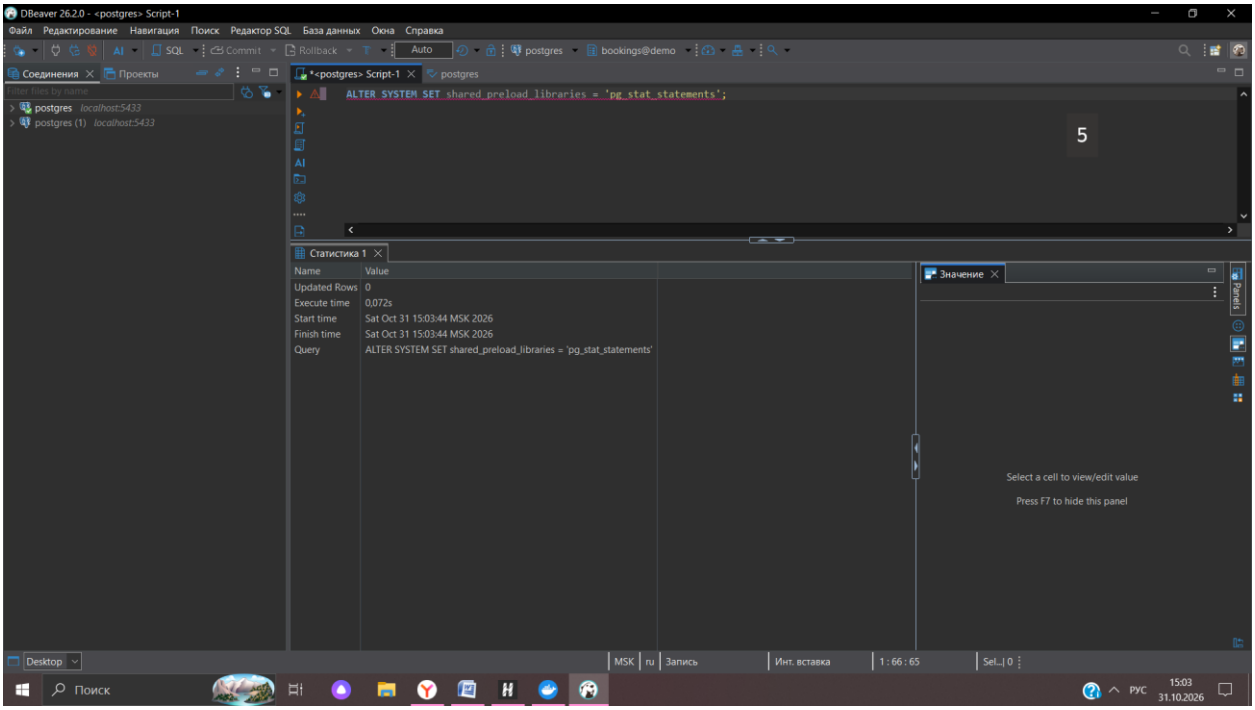
B-Tree подходит для поиска по ID и вообще для точечных запросов: быстрый поиск одной строки, первичные ключи и уникальные ограничения, сортировка (ORDER BY), выборка узких диапазонов. Он работает при любом порядке данных на диске, но занимает много места и замедляет вставку, так как дерево нужно поддерживать.

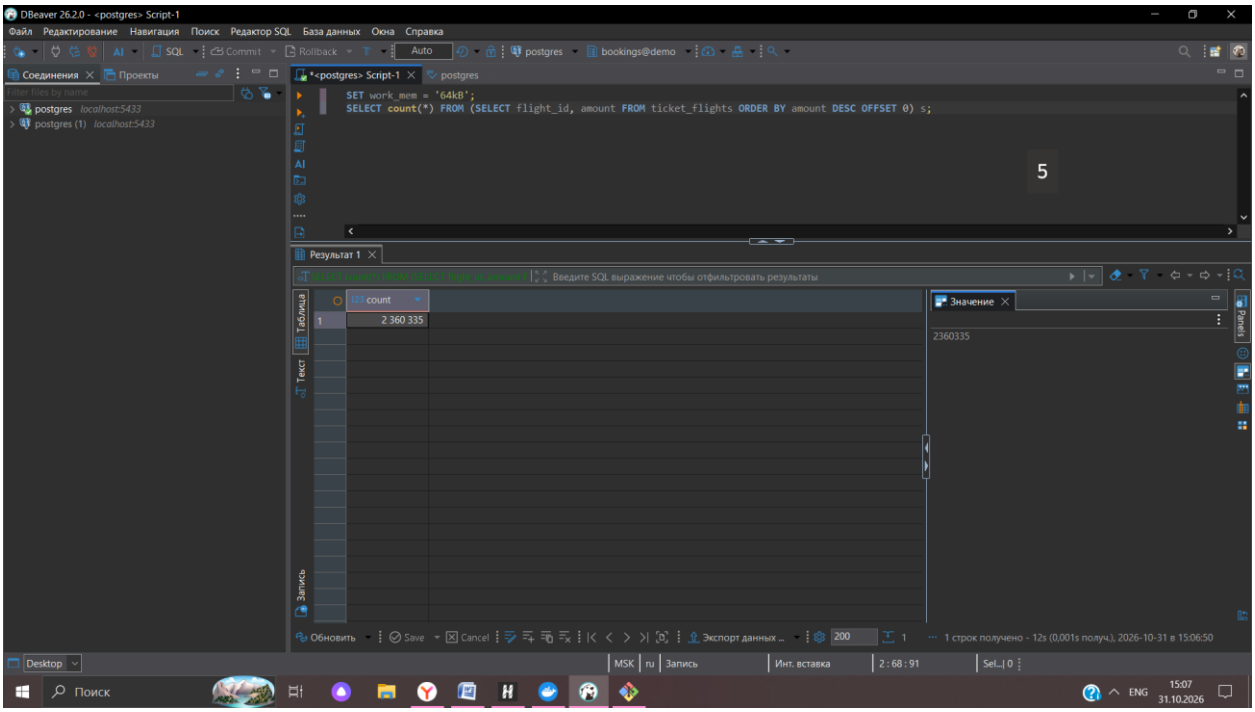
BRIN подходит для очень больших таблиц, в которые данные добавляются по порядку и физически упорядочены по столбцу, например логи или события по времени. Запросы к ним сканируют широкие диапазоны. Индекс почти не занимает места и почти не замедляет вставку. Для точечного поиска по ID он плохо подходит, а если данные на диске расположены хаотично, диапазоны перекрываются, и индекс становится почти бесполезным. Уникальность BRIN не обеспечивает.

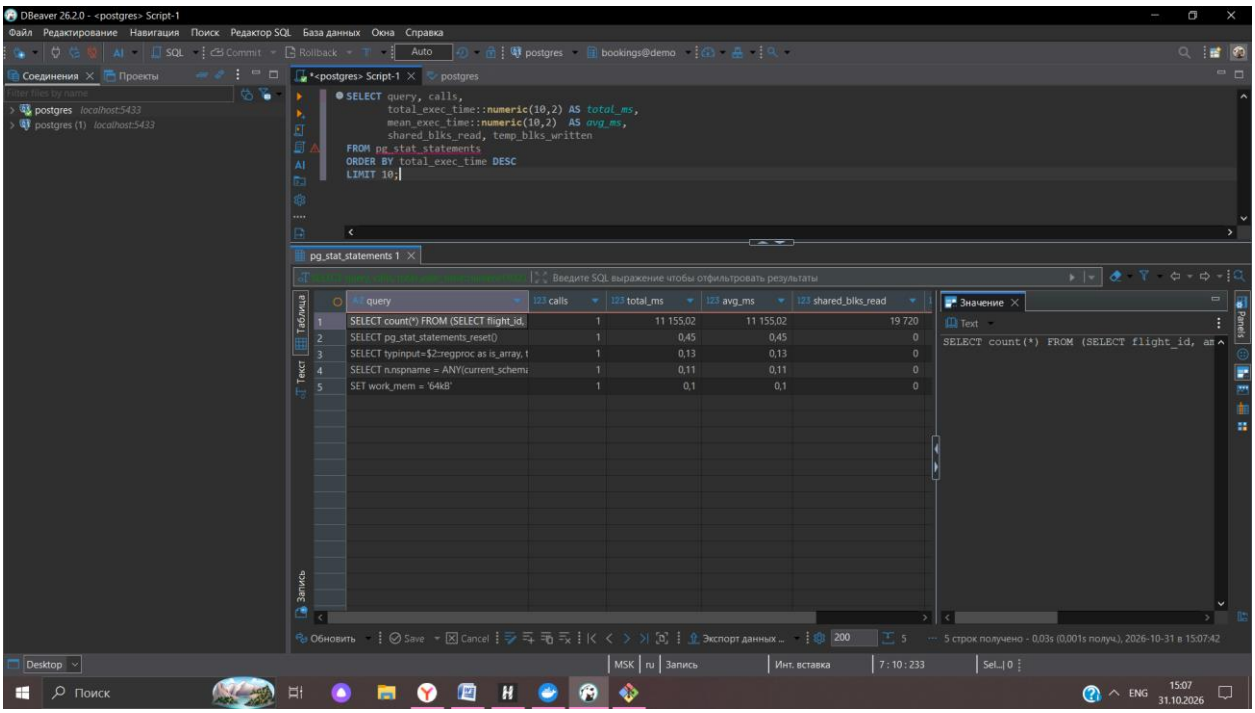
fillfactor определяет, насколько заполнять страницу при записи. Для таблицы и индексов с частыми обновлениями значение ниже 100 оставляет в страницах свободное место. Это уменьшает число разбиений страниц в B-Tree и позволяет обновлять строки внутри той же страницы без раздувания таблицы.

Партиционирование разбивает большую таблицу на небольшие части, у каждой из которых свои индексы. Запрос читает только нужные партиции, индексы получаются меньше и лучше помещаются в память, а BRIN внутри партиции работает точнее, потому что данные в ней упорядочены.

ЗАДАНИЕ 5

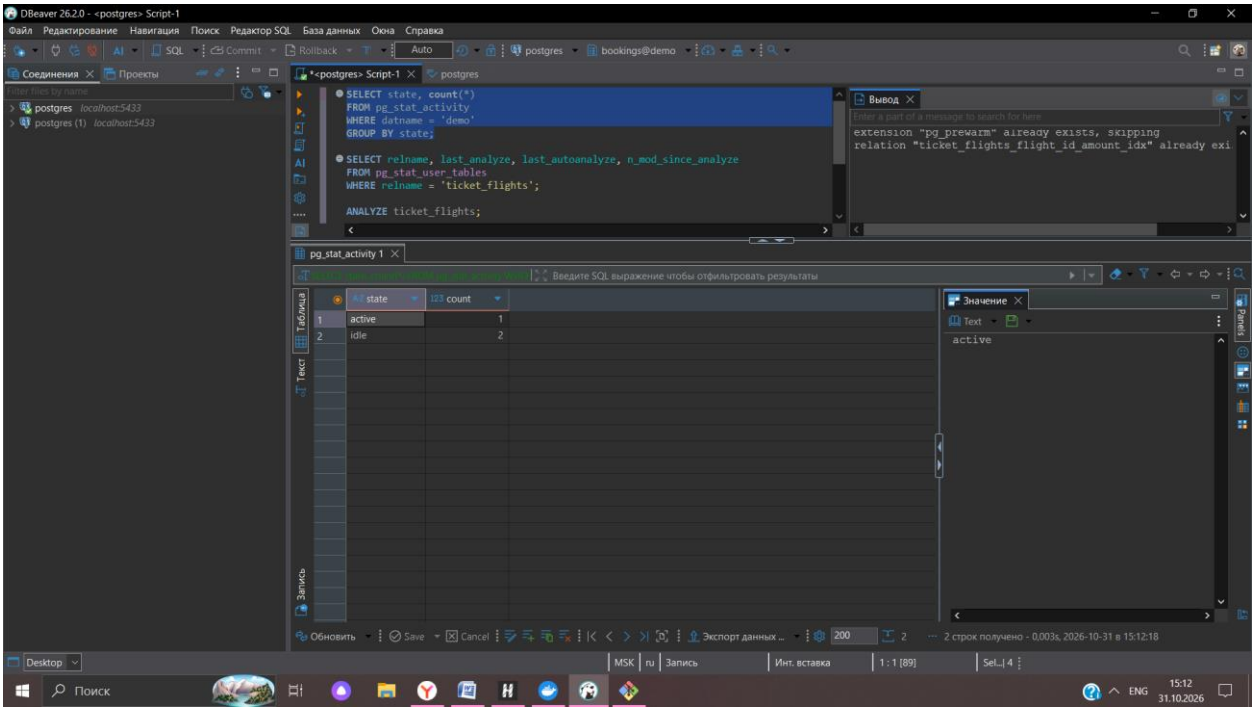


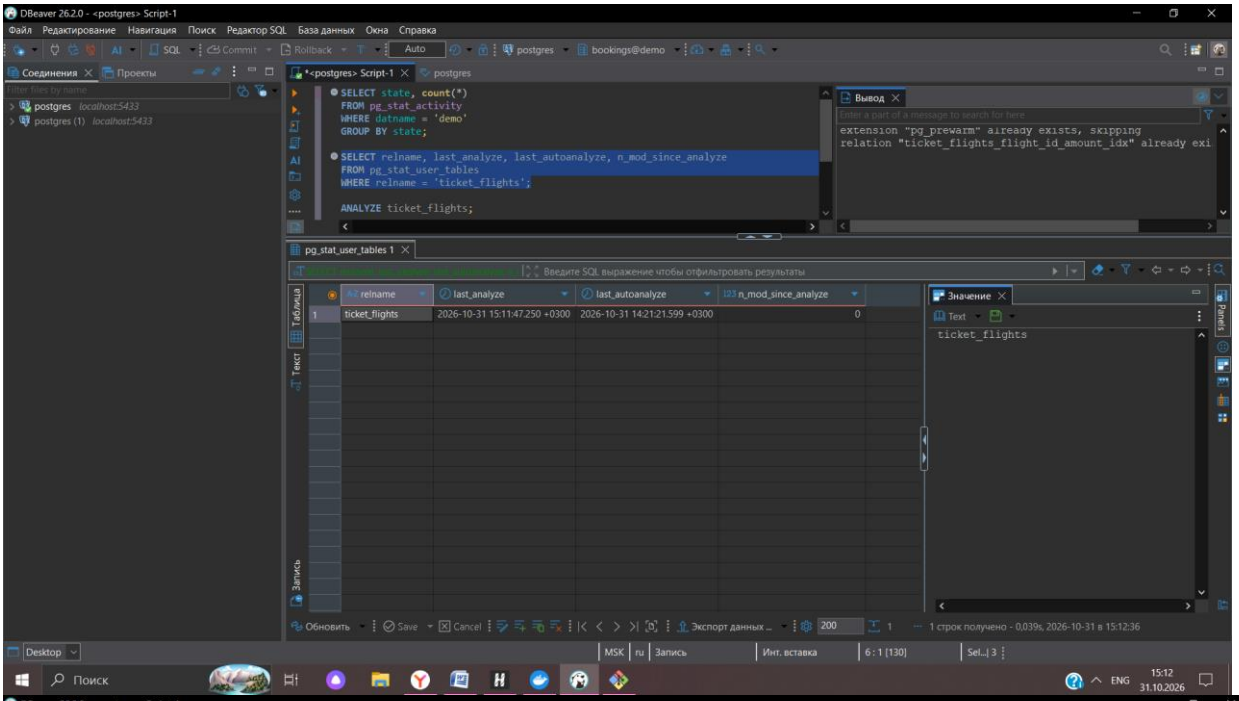


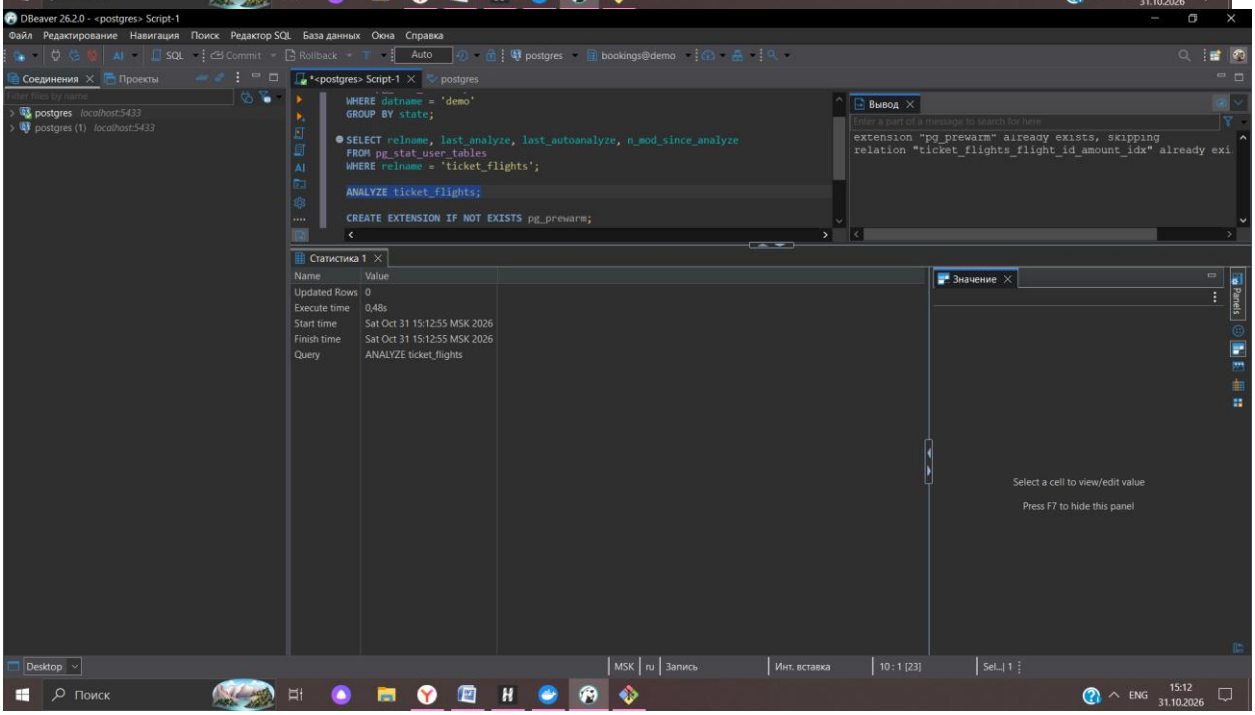


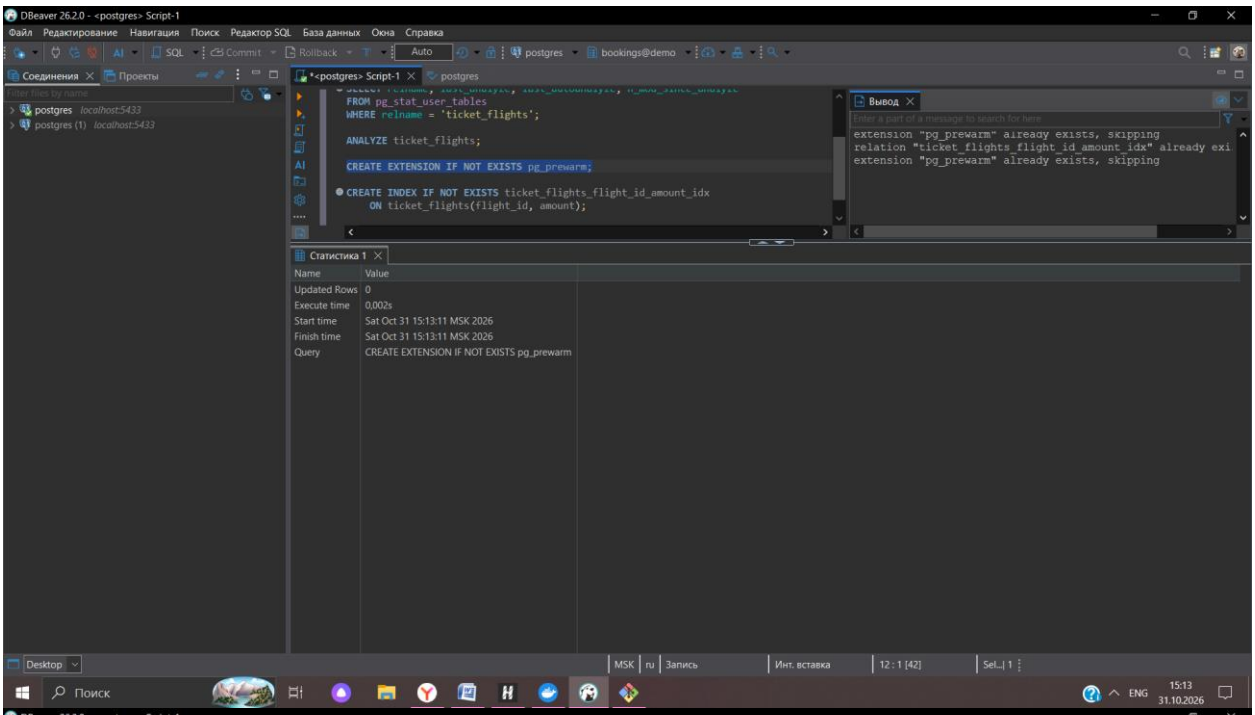
Запрос неэффективен. Значение shared_blks_read = 19 720 равно размеру всей таблицы ticket_flights в блоках: после перезапуска контейнера кэш shared_buffers пуст, и вся таблица читается с диска. Большое значение temp_blks_written говорит о том, что при work_mem = 64kB сортировка не помещается в память и промежуточные данные записываются во временные файлы, что замедляет запрос. Среднее время 11,2 с для одного вызова — слишком много.

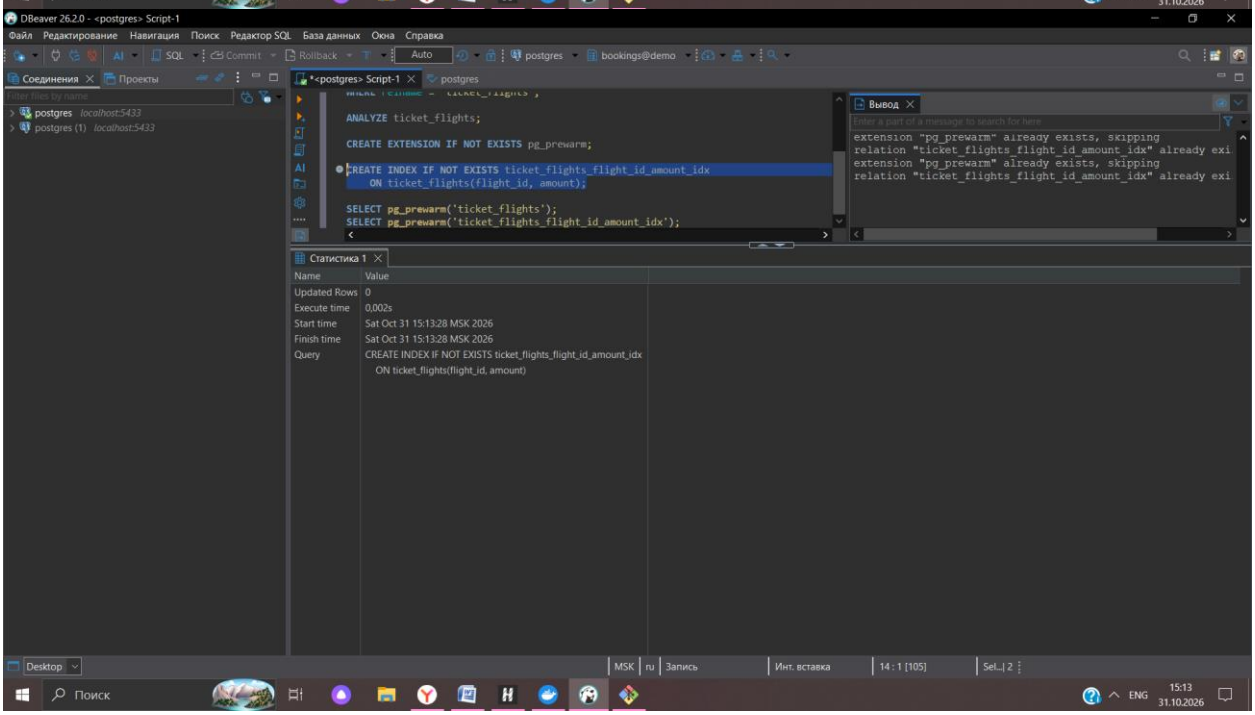
ЗАДАНИЕ 6 РАЗДЕЛ 3

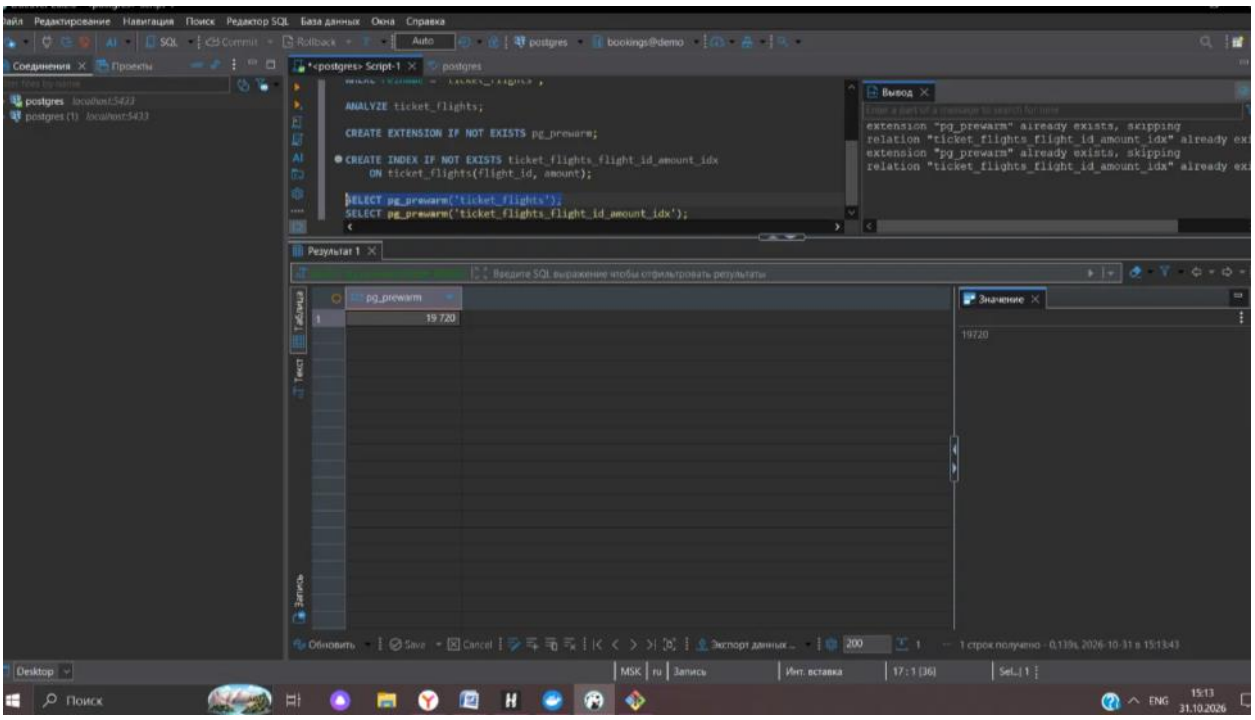


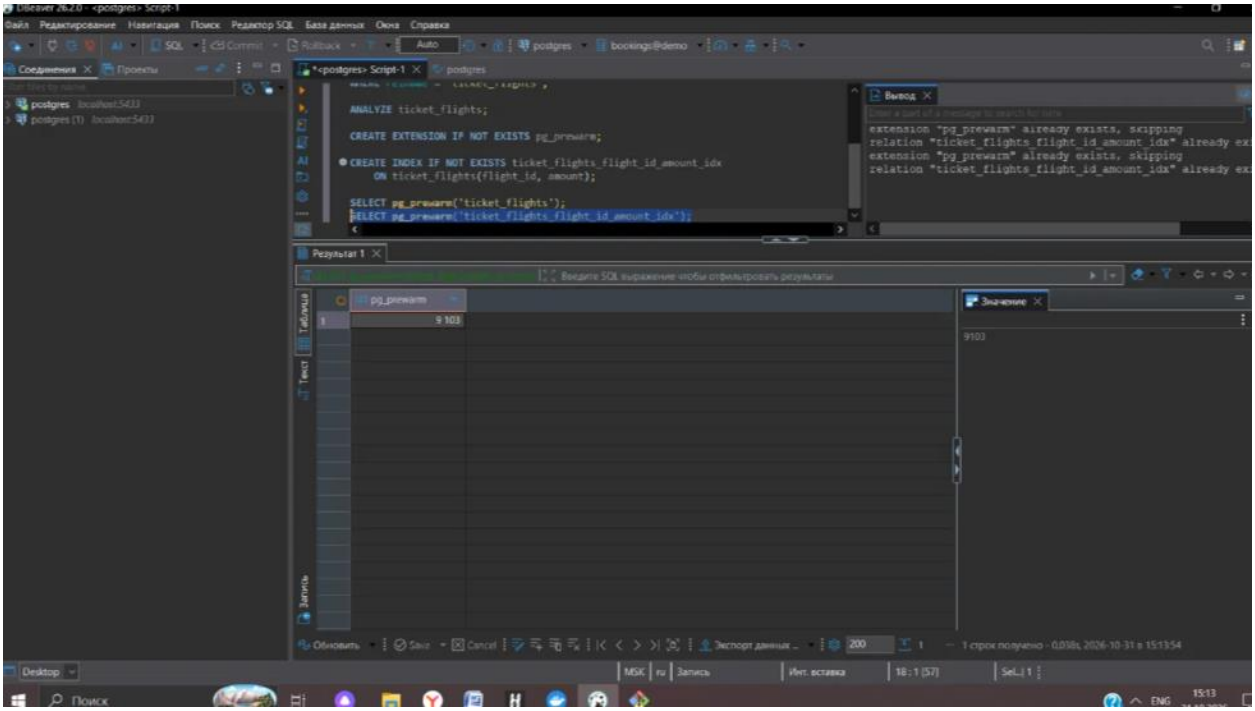

DBeaver 26.2.0 - <postgres> Script-1
Файл Редактирование Навигация Поиск Редактор SQL База данных Окна Справка
SELECT state, count(*)
FROM pg_stat_activity
WHERE datname = 'demo'
GROUP BY state;
SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
FROM pg_stat_user_tables
WHERE relname = 'ticket_flights';
ANALYZE ticket_flights;
Вывод
extension "pg_prewarm" already exists, skipping
relation "ticket_flights_flight_id_amount_idx" already exi
relname
last_analyze
last_autoanalyze
n_mod_since_analyze
ticket_flights
2026-10-31 15:11:47.250 +0300
2026-10-31 14:21:21.599 +0300
0
Значение
ticket_flights
1 строк получено - 0,039s, 2026-10-31 в 15:12:36
MSK
Запись
Инт. вставка
6 : 1 [130]
15:12
31.10.2026


DBeaver 26.2.0 - <postgres> Script-1
Файл Редактирование Навигация Поиск Редактор SQL База данных Окна Справка
WHERE datname = 'demo'
GROUP BY state;
SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
FROM pg_stat_user_tables
WHERE relname = 'ticket_flights';
ANALYZE ticket_flights;
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
Вывод
extension "pg_prewarm" already exists, skipping
relation "ticket_flights_flight_id_amount_idx" already exi
Статистика 1
Name
Value
Updated Rows
0
Execute time
0,48s
Start time
Sat Oct 31 15:12:55 MSK 2026
Finish time
Sat Oct 31 15:12:55 MSK 2026
Query
ANALYZE ticket_flights
Значение
Select a cell to view/edit value
Press F7 to hide this panel
MSK
Запись
Инт. вставка
10 : 1 [23]
15:12
31.10.2026


DBeaver 26.2.0 - <postgres> Script-1
Файл Редактирование Навигация Поиск Редактор SQL База данных Окна Справка
FROM pg_stat_user_tables
WHERE relname = 'ticket_flights';
ANALYZE ticket_flights;
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
ON ticket_flights(flight_id, amount);
extension "pg_prewarm" already exists, skipping
relation "ticket_flights_flight_id_amount_idx" already exi
extension "pg_prewarm" already exists, skipping
Updated Rows 0
Execute time 0,002s
Start time Sat Oct 31 15:13:11 MSK 2026
Finish time Sat Oct 31 15:13:11 MSK 2026
Query CREATE EXTENSION IF NOT EXISTS pg_prewarm


DBeaver 26.2.0 - <postgres> Script-1
Файл Редактирование Навигация Поиск Редактор SQL База данных Окна Справка
ANALYZE ticket_flights;
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
ON ticket_flights(flight_id, amount);
SELECT pg_prewarm('ticket_flights');
SELECT pg_prewarm('ticket_flights_flight_id_amount_idx');
extension "pg_prewarm" already exists, skipping
relation "ticket_flights_flight_id_amount_idx" already exi
extension "pg_prewarm" already exists, skipping
relation "ticket_flights_flight_id_amount_idx" already exi
Updated Rows 0
Execute time 0,002s
Start time Sat Oct 31 15:13:28 MSK 2026
Finish time Sat Oct 31 15:13:28 MSK 2026
Query CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
ON ticket_flights(flight_id, amount)





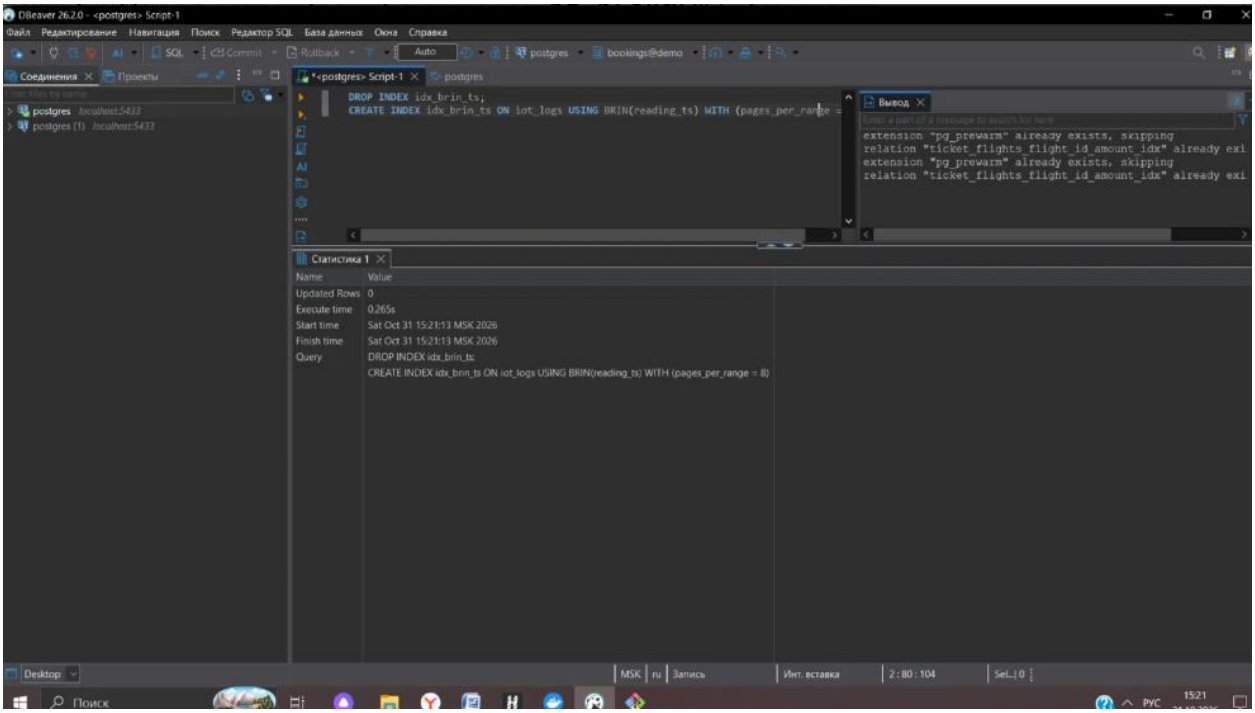
Запрос к pg_stat_activity по базе demo показал два состояния: active 1 сессия, idle 2 сессии.

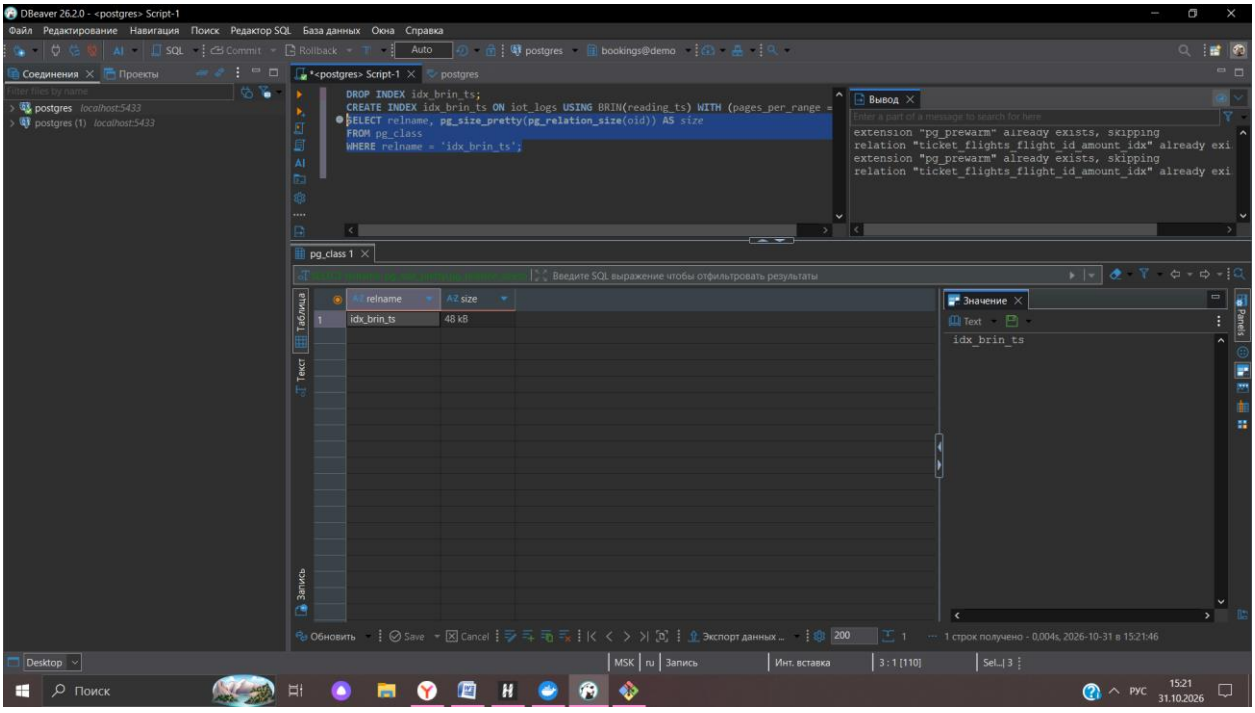
Поле state показывает, что делает каждое соединение: active — выполняет запрос, idle — подключено, но ничего не делает, idle in transaction — открыло транзакцию и ждёт. Перед замерами нужно убедиться, что кроме моей сессии на сервере нет другой нагрузки: чужие активные запросы занимают процессор и диск, вытесняют данные из кэша и искажают время выполнения EXPLAIN (ANALYZE, BUFFERS), а зависшие транзакции могут влиять на блокировки и статистику. В моём случае одна активная сессия — это сам запрос, а две простаивающие, вероятно, служебные соединения DBeaver. Сессий idle in transaction нет, значит, окружение тихое, и замеры можно считать чистыми.

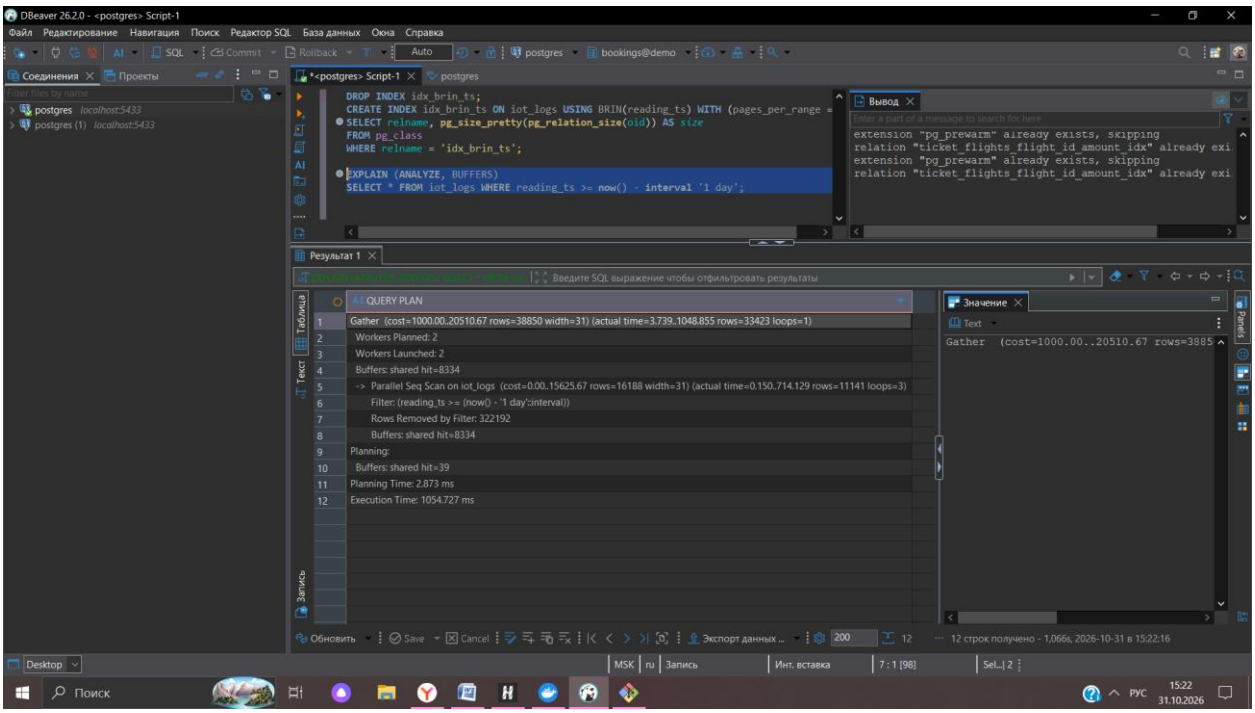
В кэш загружено 19 720 блоков (около 154 МБ) и загружено 9 103 блока (около 71 МБ).

pg_prewarm заранее читает таблицу и индекс в буферный кэш shared_buffers. Тогда первый замер не включает медленное чтение с диска, и результаты сравнений не зависят от того, был ли кэш холодным. Число возвращённых блоков показывает, сколько блоков прочитано, но не гарантирует, что все они остались в кэше: вместе таблица и индекс занимают около 225 МБ, и если shared_buffers меньше, часть блоков будет вытеснена.

РАЗДЕЛ 4 ПРАКТИКУМ 3







BRIN-индекс хранит не ссылки на каждую строку, а минимальные и максимальные значения для диапазонов страниц. Параметр pages_per_range определяет, сколько страниц таблицы объединяется в один диапазон.

При pages_per_range = 128: В один блок индекса объединяется 128 страниц (около 1 МБ данных). Индекс получается очень компактным, так как содержит мало записей о диапазонах.

При pages_per_range = 8: Количество страниц в диапазоне уменьшается. Это значит, что для той же таблицы создается больше диапазонов. Как видно из скриншота, размер индекса увеличился до 40 kB. Чем меньше страниц в диапазоне, тем больше метаданных (минимумов и максимумов) нужно хранить индексу, поэтому он занимает больше места.
Параметр pages_per_range создает баланс между размером индекса и точностью поиска.

При большом значении (128) индекс быстрее отсекает заведомо неподходящие данные, но внутри найденного диапазона PostgreSQL придется просматривать больше страниц и проверять каждую строку.

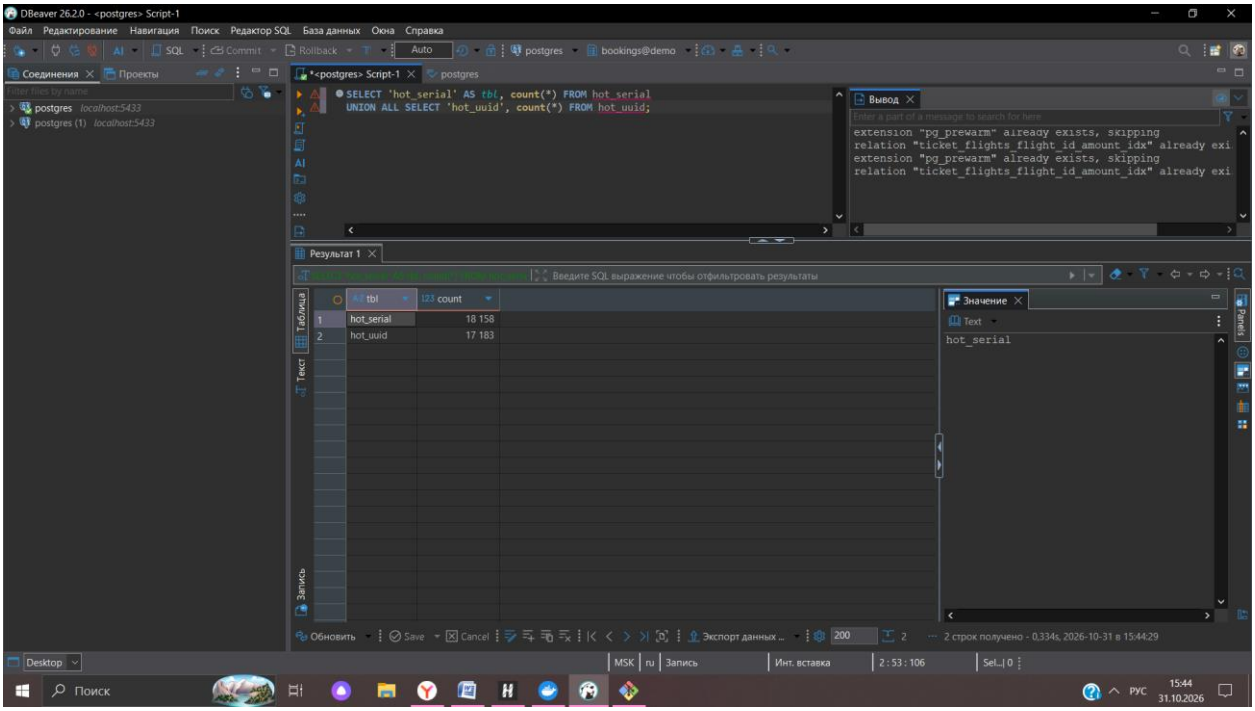
При маленьком значении (8) точность выше. Индекс точнее указывает на нужные страницы, и PostgreSQL читает меньше лишних данных с диска. Однако за это приходится платить чуть большим размером индекса (40 kB против минимального).

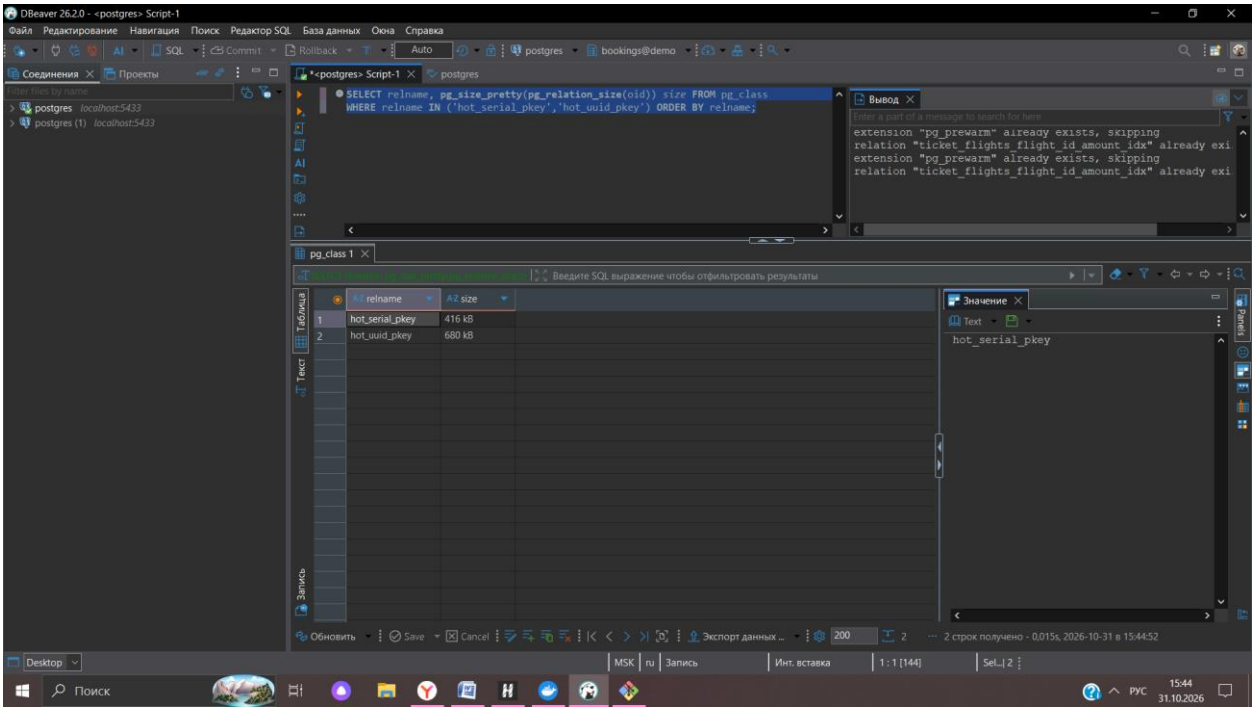
Раздел 6. Практикум 4

```
=== serial ===
number of transactions actually processed: 18158
tps = 906.743630 (without initial connection time)
      63  LWLock|WALWrite
      10  IO|WALSync
       2  |
       2  Client|ClientRead
=== uuid ===
number of transactions actually processed: 17183
```

```
tps = 859.277302 (without initial connection time)
      59  LWLock|WALWrite
       9  IO|WALSync
       8  |
       1  IO|WALWrite
       1  Client|ClientRead
    tbl     | count
------------+-------
 hot_serial | 18158
 hot_uuid   | 17183
(2 rows)

     relname     |  size
-----------------+--------
 hot_serial_pkey | 416 kB
 hot_uuid_pkey   | 680 kB
(2 rows)
```

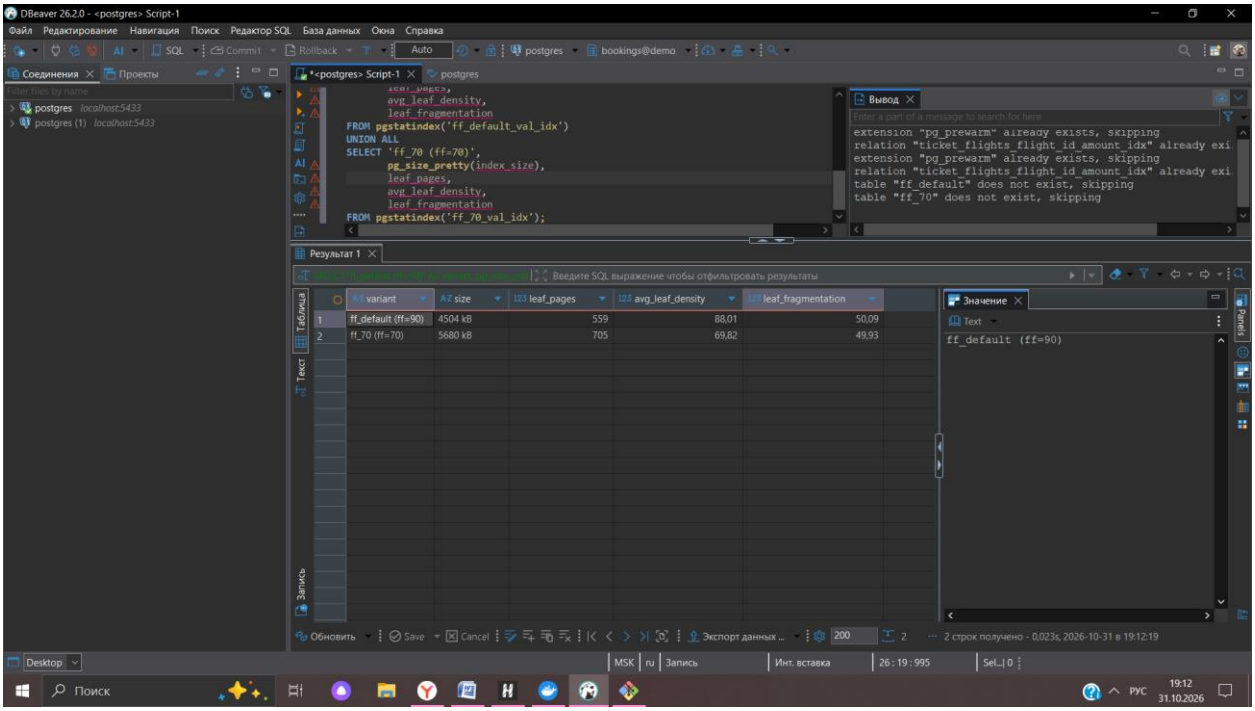




При сравнении топов wait events из логов pgbench видно, что в обоих случаях доминирует ожидание LWLock:WALWrite (63 раза для hot_serial и 59 раз для hot_uuid), что связано с необходимостью сброса журнала предзаписи на диск при интенсивных вставках. При использовании hot_serial все параллельные потоки пытаются вставить новые строки в конец B-Tree индекса, так как значения BIGSERIAL строго возрастают. В случае с hot_uuid значения генерируются случайно, поэтому вставки распределяются равномерно по всему пространству индекса, и потоки не мешают друг другу писать в одну и ту же страницу.

Утилита pgbench представляет собой встроенный в PostgreSQL инструмент для нагрузочного тестирования. Она имитирует работу множества одновременных клиентов (пользователей), каждый из которых выполняет заданный SQL-скрипт (в данном случае команду INSERT). pgbench запускает эти процессы параллельно в течение заданного времени и по завершении выводит статистику: сколько всего транзакций было успешно обработано и сколько транзакций в секунду (TPS) смогла выполнить база данных. Это позволяет на практике оценить, как конкретные архитектурные решения (например, тип первичного ключа) влияют на производительность системы под нагрузкой.

РАЗДЕЛ 7 ПАРКТИКУМ 5



Параметр avg_leaf_density показывает, насколько заполнены полезными данными листовые узлы индекса. Значение 100 означает, что страница заполнена полностью, а 50 — что заполнена только половина. У меня плотность индекса ff_default составляет 88.01%, а у ff_70 — 69.82%.

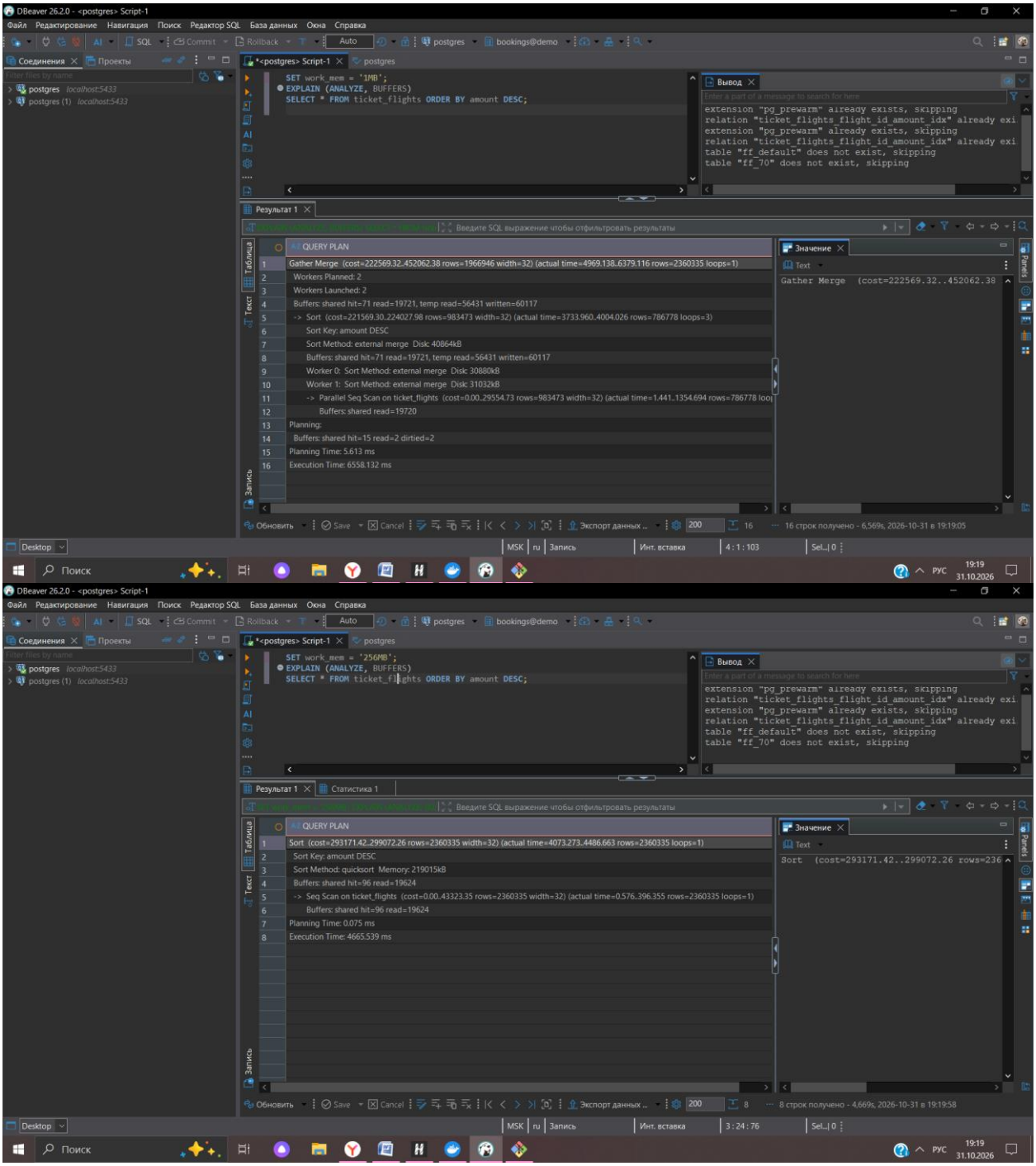
Параметр leaf_fragmentation показывает процент страниц индекса, которые физически расположены в файле не по порядку относительно их логического порядка в B-Tree. Чем выше фрагментация, тем больше операций ввода-вывода требуется для последовательного чтения индекса. Фрагментация у обоих индексов примерно одинакова (около 50%), так как вставка случайных значений изначально создает хаотичную структуру.

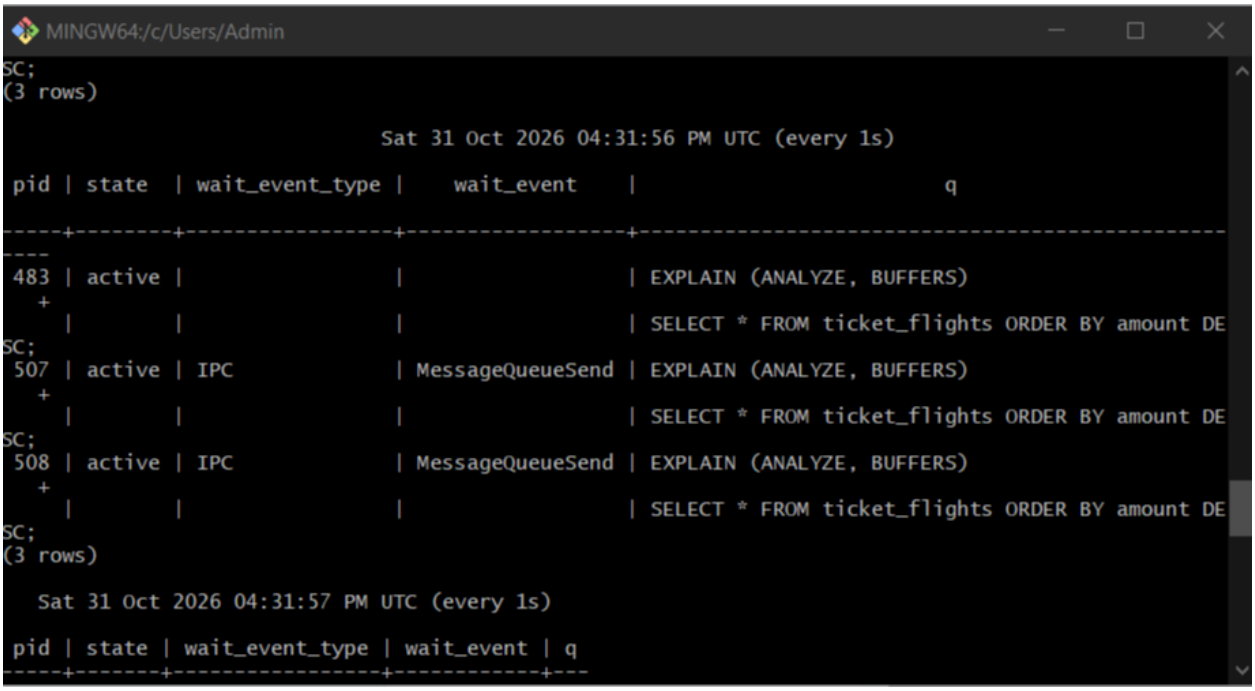
Понижение fillfactor до 70 напрямую влияет на процесс разделения страниц. При стандартном значении 90 страница заполняется почти полностью, и когда в такой плотный индекс вставляются

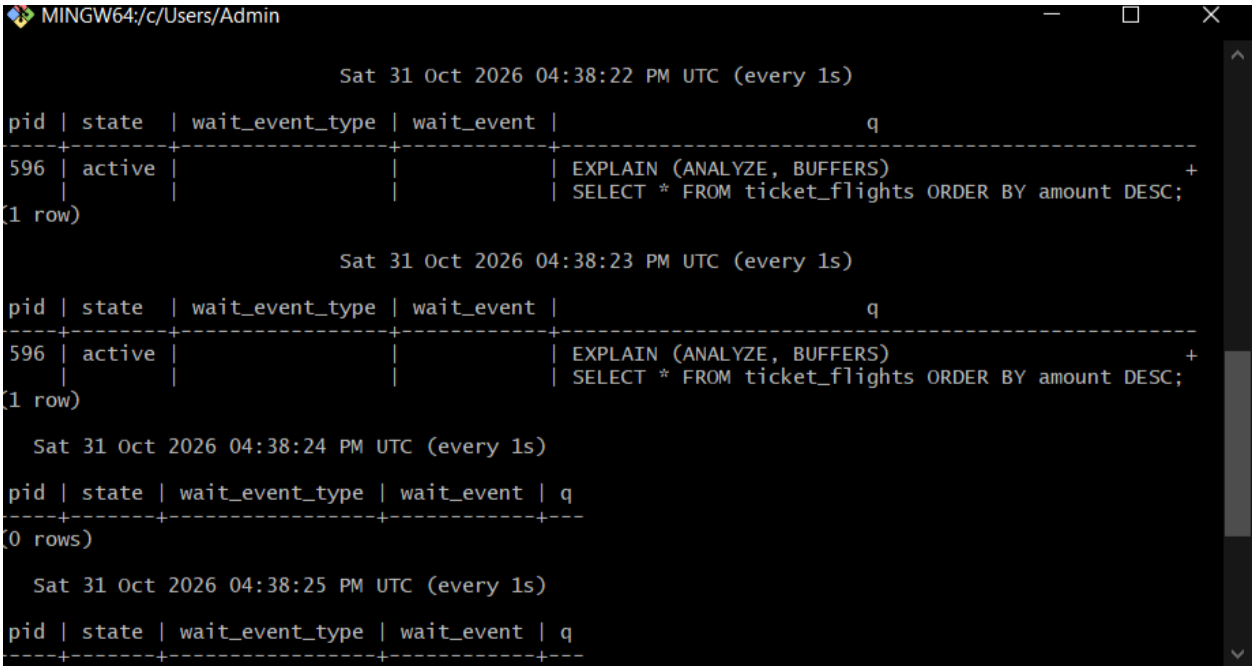
новые случайные значения, PostgreSQL вынужден часто разбивать переполненные страницы на две. При fillfactor = 70 в страницах индекса изначально оставляется 30% свободного места. Благодаря этому новые случайные вставки чаще помещаются прямо в существующие страницы без разбиения, что снижает количество Page Splits.

Задавать индекс с пониженным fillfactor целесообразно для таблиц, в которых очень часто происходят операции вставки и обновления в случайные места. Это помогает избежать частых Page Splits и снижает фрагментацию. Однако для таблиц, в которые данные в основном только добавляются в конец (по возрастающему ключу) или которые редко обновляются, понижать fillfactor не нужно, так как это приведет к искусственному увеличению размера индекса без реальной пользы.

Раздел 8. Практикум 6

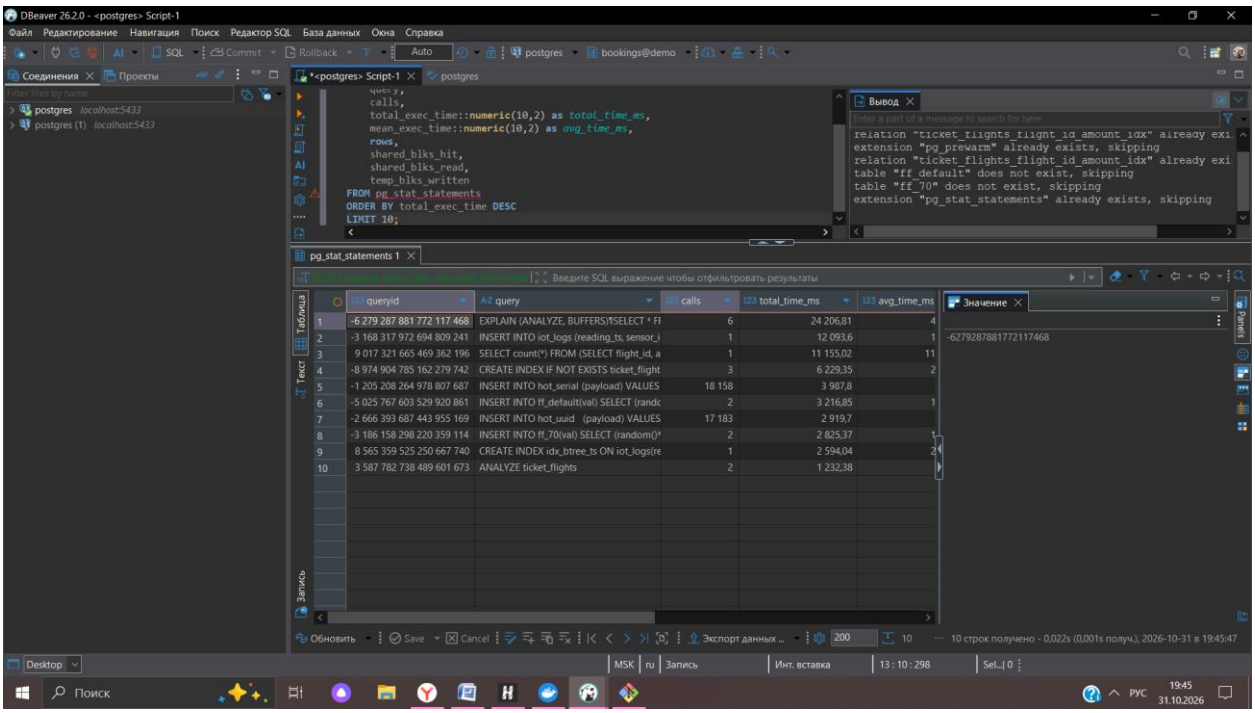


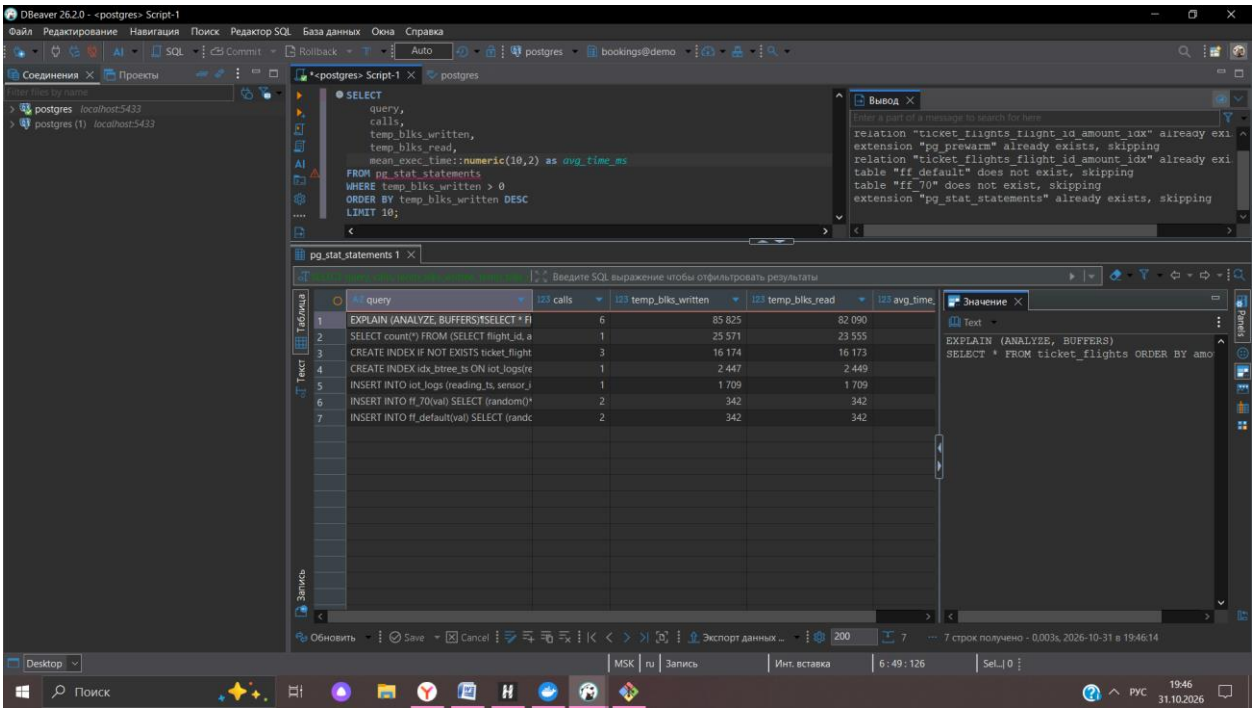


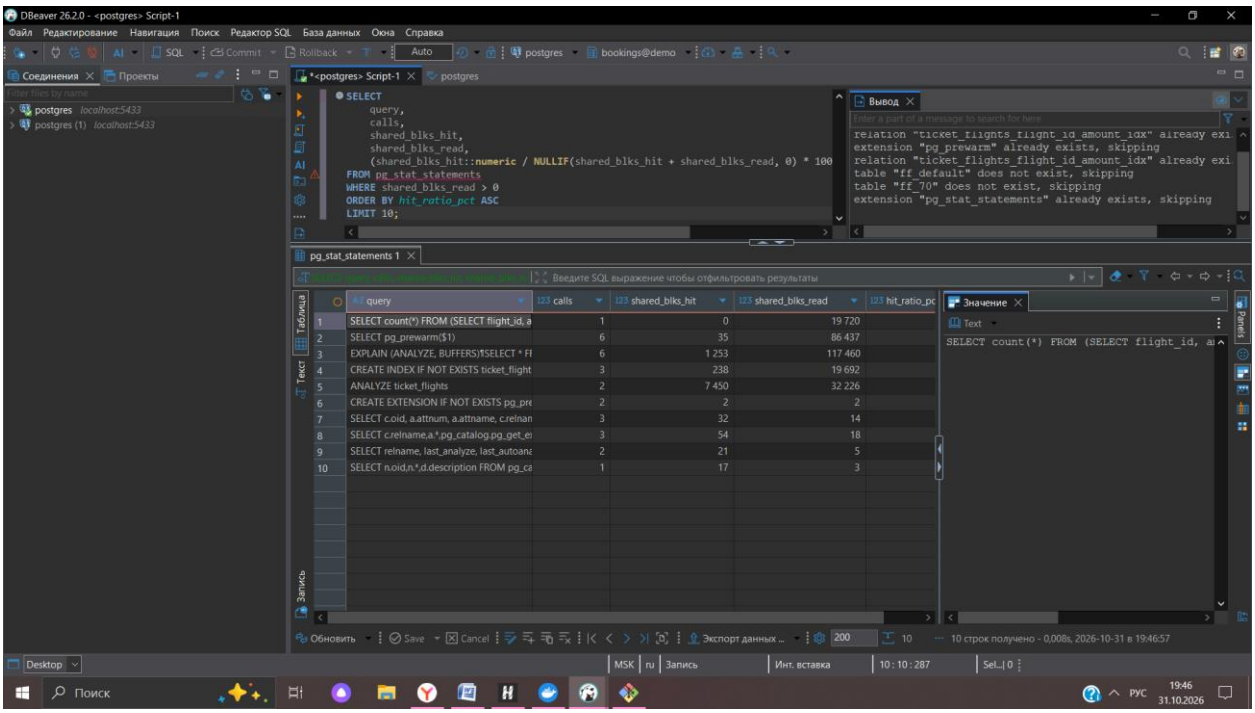


Зафиксированы события ожидания (Wait Events) для двух режимов работы параметра work_mem. На первом скриншоте, соответствующем режиму work_mem = '1MB', видно, что рабочие процессы находятся в ожидании IPC:MessageQueueSend. Это связано с тем, что главный процесс был занят сбросом промежуточных данных на диск из-за нехватки памяти, что подтверждается методом сортировки Sort Method: external merge Disk в плане запроса. На втором скриншоте, соответствующем режиму work_mem = '256MB', видно, что процесс активен, но поля wait_event_type и wait_event пусты. Это означает, что база больше не ждет операций ввода-вывода, так как вся сортировка выполняется в оперативной памяти. Это полностью сопоставляется с изменением метода сортировки в плане на Sort Method: quicksort Memory, где все данные поместились в выделенные 256MB, и время выполнения запроса сократилось.

Раздел 9


DBeaver 26.2.0 - <postgres> Script-1
Файл Редактирование Навигация Поиск Редактор SQL База данных Окна Справка
Соединения Проекты
postgres localhost:5433
postgres (1) localhost:5433
*<postgres> Script-1
total_exec_time::numeric(10,2) as total_time_ms,
mean_exec_time::numeric(10,2) as avg_time_ms,
rows,
shared_blks_hit,
shared_blks_read,
temp_blks_written
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
Вывод
relation "ticket_flights_flight_id_amount_idx" already exi
extension "pg_prewarm" already exists, skipping
relation "ticket_flights_flight_id_amount_idx" already exi
table "ff_default" does not exist, skipping
table "ff_70" does not exist, skipping
extension "pg_stat_statements" already exists, skipping
pg_stat_statements 1
Введите SQL выражение чтобы отфильтровать результаты
queryid query calls total_time_ms avg_time_ms
-6 279 287 881 772 117 468 EXPLAIN (ANALYZE, BUFFERS)\SELECT * FI 6 24 206,81 4
-3 168 317 972 694 809 241 INSERT INTO iot_logs (reading_ts, sensor_i 1 12 093,6 1
9 017 321 665 469 362 196 SELECT count(*) FROM (SELECT flight_id, a 1 11 155,02 11
-8 974 904 785 162 279 742 CREATE INDEX IF NOT EXISTS ticket_flight 3 6 229,35 2
-1 205 208 264 978 807 687 INSERT INTO hot_serial (payload) VALUES 18 158 3 987,8
-5 025 767 603 529 920 861 INSERT INTO ff_default(val) SELECT (randc 2 3 216,85 1
-2 666 393 687 443 955 169 INSERT INTO hot_uuid (payload) VALUES 17 183 2 919,7
-3 186 158 298 220 359 114 INSERT INTO ff_70(val) SELECT (random()* 2 2 825,37 1
8 565 359 525 250 667 740 CREATE INDEX idx_btree_ts ON iot_logs(re 1 2 594,04 2
3 587 782 738 489 601 673 ANALYZE ticket_flights 2 1 232,38
Значение
-6279287881772117468
Обновить Save Cancel Экспорт данных ... 200 10
10 строк получено - 0,022s (0,001s получ.), 2026-10-31 в 19:45:47
Desktop
MSK ru Запись Инт. вставка 13 : 10 : 298 Sel...| 0
Поиск
РУС 19:45 31.10.2026


DBeaver 26.2.0 - <postgres> Script-1
Файл Редактирование Навигация Поиск Редактор SQL База данных Окна Справка
Соединения Проекты
postgres localhost:5433
postgres (1) localhost:5433
*<postgres> Script-1
SELECT
query,
calls,
temp_blks_written,
temp_blks_read,
mean_exec_time::numeric(10,2) as avg_time_ms
FROM pg_stat_statements
WHERE temp_blks_written > 0
ORDER BY temp_blks_written DESC
LIMIT 10;
Вывод
relation "ticket_flights_flight_id_amount_idx" already exi
extension "pg_prewarm" already exists, skipping
relation "ticket_flights_flight_id_amount_idx" already exi
table "ff_default" does not exist, skipping
table "ff_70" does not exist, skipping
extension "pg_stat_statements" already exists, skipping
pg_stat_statements 1
Введите SQL выражение чтобы отфильтровать результаты
query calls temp_blks_written temp_blks_read avg_time_
EXPLAIN (ANALYZE, BUFFERS)\SELECT * FI 6 85 825 82 090
SELECT count(*) FROM (SELECT flight_id, a 1 25 571 23 555
CREATE INDEX IF NOT EXISTS ticket_flight 3 16 174 16 173
CREATE INDEX idx_btree_ts ON iot_logs(re 1 2 447 2 449
INSERT INTO iot_logs (reading_ts, sensor_i 1 1 709 1 709
INSERT INTO ff_70(val) SELECT (random()* 2 342 342
INSERT INTO ff_default(val) SELECT (randc 2 342 342
Значение
Text
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_flights ORDER BY amo
Обновить Save Cancel Экспорт данных ... 200 7
7 строк получено - 0,003s, 2026-10-31 в 19:46:14
Desktop
MSK ru Запись Инт. вставка 6 : 49 : 126 Sel...| 0
Поиск
РУС 19:46 31.10.2026



В топе самых медленных запросов (по total_exec_time) первое место занимает тяжелый аналитический запрос EXPLAIN (ANALYZE, BUFFERS) SELECT... с общим временем выполнения 24 206 мс и средним временем 4 034 мс. На втором и третьем местах находятся запросы SELECT из таблицы iot_logs и SELECT count(*), что говорит о том, что операции с большими таблицами являются самыми ресурсоемкими в текущей базе.

Анализ запросов со сбросом на диск (по temp_blks_written) показывает, что тот же самый запрос EXPLAIN (ANALYZE, BUFFERS) SELECT... лидирует с показателем 65 825 записанных временных блоков. Это напрямую связано с тем, что при выполнении этого запроса использовался параметр work_mem = '1MB', и памяти не хватило для сортировки, из-за чего PostgreSQL был вынужден сбрасывать промежуточные данные во временные файлы на диске. Остальные запросы также пишут временные блоки, но в гораздо меньших объемах.

Третий запрос показывает список запросов с самым низким коэффициентом попадания в кэш (cache hit ratio). Запрос SELECT count(*) имеет hit_ratio_pct = 0, так как он прочитал 19 720 блоков с диска и ни одного из кэша — это означает, что данные не были предварительно загружены в shared_buffers. Запрос SELECT pg_prewarm($1) также показывает низкий процент, что логично, так как его задача — как раз загрузить данные с диска в память. Самые эффективные запросы в этом списке — это EXPLAIN и ANALYZE с высокими показателями shared_blks_hit (например, 7 450 против 32 226 прочитанных), что говорит о хорошем использовании оперативной памяти при их выполнении.

Общий вывод: расширение pg_stat_statements позволяет точно определить, какие запросы нагружают систему больше всего. В нашем случае главным источником медленной работы является тяжелый SELECT с сортировкой, который не помещается в work_mem и уходит на диск.

7. Теоретическое задание: назначение системных представлений статистики

№	Объект	1. Что это за объект и в какой схеме/расширении живёт	2. Ключевые столбцы (что означает каждый)	3. Что показывает значение (единицы, диапазон, накопление/срез)	4. Для чего нужен	5. Пример использования из ДЗ	6. Ограничения / подводные камни
1	pg_stat_user_tables	Системное представление (view) в схеме pg_catalog.	last_analyze — время последнего ручного ANALYZE; last_autoanalyze — время последнего автоанализа; n_mod_since_analyze — число строк, изменённых с момента последнего анализа; seq_scan — кол-во последовательных сканирований; idx_scan — кол-во сканирований по индексу; n_live_tup — живые строки; n_dead_tup — мертвые	Время (timestamp), количество (штуки). Счётчики накапливаются, n_mod_since_analyze сбрасывается в 0 после ANALYZE.	Диагностика устаревания статистики и необходимости ANALYZE. Понимание соотношения seq/idx сканов. Оценка необходимости VACUUM.	Задание 1: запрос к flights показал last_autoanalyze свежий, n_mod_since_analyze = 0, что выявило отсутствие необходимости в ручном ANALYZE.	Показывает срез с момента сброса, а не историю. n_live_tup и n_dead_tup — приблизительные оценки.

			строки.				
2	pg_stat_activity	Системное представление (view) в схеме pg_catalog.	state — состояние (active, idle); wait_event_type — тип ожидания (IO, Lock, IPC); wait_event — конкретное событие; query — текст запроса; query_start — время начала; backend_type — тип процесса.	Время, текст, состояние. Показывает текущий срез на момент запроса.	Диагностика "прямо сейчас": поиск долгих запросов, блокировок, понимание, чего именно ждёт база.	Задание 8: использовали для фиксации IPC:MessageQueueSend при work_mem=1MB и пустых ожиданий при work_mem=256MB.	Это срез, а не история. Если запрос завершился, вы его не увидите. query может обрезаться.
3	pg_stat_statements	Расширение (extension), создаётся через CREATE EXTENSION.	queryid — ID нормализованного запроса; calls — кол-во вызовов; total_exec_time — общее время (мс); mean_exec_time — среднее время (мс); shared_blks_hit — блоки из кэша; shared_blks_read — блоки с диска; temp_blks_written — временные блоки на диск; rows —	Миллисекунды, блоки (8КБ), штуки. Накопительные с момента сброса.	Поиск самых медленных запросов, запросов с плохим кэшированием и тех, что уходят на диск. Тюнинг work_mem.	Задание 5: нашли EXPLAIN... SELECT на первом месте по total_exec_time и temp_blks_written.	Не видит запросы до загрузки shared_preload_libraries. Нормализует литералы (заменяет константы на $1).

			возвращённые строки.				
4	pg_class	Системный каталог (таблица) в схеме pg_catalog.	relname — имя объекта; relkind — тип (r=таблица, i=индекс); relpages — кол-во страниц; reltuples — кол-во строк; pg_relation_size() — размер; pg_total_relation_size() — полный размер.	Байты (для функций), страницы (8КБ), штуки.	Мониторинг размеров объектов, оценка "веса" таблиц и индексов, сравнение эффективности индексов.	Задания 4, 6, 7: сравнивали размеры B-Tree и BRIN, а также индексов с разным fillfactor.	relpages и reltuples обновляются только после ANALYZE/VACUUM и могут быть устаревшими.
5	pg_prewarm	Расширение (extension), создаётся через CREATE EXTENSION.	Функция pg_prewarm() принимает имя таблицы/индекса, параметры mode (prefetch, read, buffer) и fork (main, fsm, vm).	Количество загруженных блоков (штуки).	Прогрев кэша перед важными запросами, чтобы избежать холодного чтения с диска.	Задание 2: после pg_prewarm('flights') в плане shared read сменилось на shared hit.	Не гарантирует, что данные останутся в кэше навсегда. Не стоит прогревать редкие таблицы.
6	pgstattuple / pgstatindex	Расширения (extensions), создаются через CREATE EXTENSION.	avg_leaf_density — средняя плотность листьев (%); leaf_fragmentation — фрагментация (%); leaf_pages — кол-во листовых страниц; index_size — размер индекса; free_pe	Проценты (%), байты, штуки.	Диагностика фрагментации индексов, оценка эффективности fillfactor, анализ физического состояния.	Задание 7: сравнили avg_leaf_density и leaf_fragmentation для индексов с fillfactor 90 и 70.	Функции могут быть ресурсоёмкими (читают все страницы). pgstattuple блокирует таблицу.

			rcent — свободное место (%).				
7	pg_stat_bgwriter	Системное представление (view) в схеме pg_catalog.	checkpoints_timed — плановые чекпоинты; checkpoints_req — принудительные; buffers_backend — записано бэкендами; buffers_checkpoint — записано при чекпоинтах; buffers_alloc — выделено буферов.	Штуки (блоки, чекпоинты). Накопительные с момента старта сервера.	Понимание распределения нагрузки на диск. Диагностика частых checkpoints_req (нехватка max_wal_size).	В ДЗ напрямую не использовался.	В PostgreSQL 16+ появилось более детальное pg_stat_io.
8	pg_stat_database	Системное представление (view) в схеме pg_catalog.	blks_hit — блоки из кэша; blks_read — блоки с диска; tup_returned — возвращённые строки; tup_fetched — извлечённые по индексу строки; xact_commit — зафиксированные транзакции; xact_rollback — откаты.	Штуки (блоки, строки, транзакции). Накопительные с момента сброса.	Общая оценка нагрузки на базу, коэффициента попадания в кэш, выявление аномалий.	В ДЗ напрямую не использовался.	Включает все операции, включая служебные. Для точной диагностики лучше pg_stat_statements.

В ходе выполнения домашнего задания наиболее полезными оказались расширение pg_stat_statements и представление pg_stat_activity. pg_stat_statements позволило точно определить самые медленные запросы и запросы, уходящие на диск (по temp_blks_written), что дало возможность целенаправленно оптимизировать work_mem. pg_stat_activity оказалось незаменимым для диагностики в данный момент, оно наглядно показало, чего именно ждёт база (диск или процессор) в момент выполнения тяжёлого запроса. При следующей диагностике аналогичного стенда я бы в первую очередь проверяла топ медленных запросов через pg_stat_statements и запросы с temp_blks_written > 0, а затем смотрела в pg_stat_activity, чтобы понять, чего именно ждёт база в момент нагрузки